Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

M3dmap functions

Synopsis

The functions prefixed with M3dmap make up the Aurora Imaging Library 3D Reconstruction module. The Aurora Imaging Library 3D Reconstruction module can be used to obtain corrected information from a laser line profiling setup. Such a setup consists of a device projecting a sheet of light, usually a laser diode, coupled with a camera. When an object moves under the laser plane, the camera takes images of the intersection of that plane with the object. This results in a set of images containing a laser line, each line representing a "slice" of the object being scanned. This module can also perform stereo vision triangulation if you use two or more cameras. Triangulation calculates the X-, Y-, and Z-world coordinate of a point from the point's X and Y coordinates on the image plane of each camera.

Functions

M3dmapAddScan
M3dmapAlignScan
M3dmapAlloc
M3dmapAllocResult
M3dmapCalibrate
M3dmapCalibrateMultiple
M3dmapClear
M3dmapControl
M3dmapCopy
M3dmapCopyResult
M3dmapDraw
M3dmapDraw3d
M3dmapFree
M3dmapGetResult
M3dmapInquire
M3dmapRestore
M3dmapSave
M3dmapStream
M3dmapTriangulate

M3dmapAddScan

Synopsis

Extract laser line data from an image and store this data in a 3D reconstruction result buffer.

Syntax

```
void M3dmapAddScan(
    AIL_ID LaserContext3dmapId,       //in
    AIL_ID Result3dmapId,             //out
    AIL_ID LaserOrDepthMapImageBufId, //in
    AIL_ID IntensityImageBufId,       //in
    AIL_ID ExtraInfoArrayId,          //in
    AIL_INT PointCloudLabel,          //in
    AIL_INT64 ControlFlag             //in
)
```

Description

This function stores laser line data extracted from grabbed images into the specified result buffer.

If the specified result buffer is of type M_LASER_CALIBRATION_DATA, you can use this data to calibrate a laser reconstruction setup using M3dmapCalibrate() or M3dmapCalibrateMultiple(). If your camera has integrated hardware capable of extracting the laser line data, call **M3dmapAddScan**() with M_LINE_ALREADY_EXTRACTED to skip the extraction process and store laser line data in the result buffer.

If the number of extracted laser lines is greater than the maximum number of lines that can be stored in the result buffer, specified using M3dmapControl() with M_MAX_FRAMES, the result buffer's oldest values are overwritten with the new ones.

When specifying a 3D reconstruction result buffer allocated using M_POINT_CLOUD_RESULT, calls to **M3dmapAddScan**() either create a new point cloud in the result buffer and add laser line data to it, or just add laser line data to the specified point cloud in the result buffer. To create a new point cloud, specify a point cloud label that does not currently exist in the specified 3D reconstruction result buffer.

Between each set of scans for different objects or for cumulative extraction mode, you can clear the content of the result buffer using M3dmapClear().

> Note that the size and depth of the source image buffer containing the laser lines must not change when making multiple calls to **M3dmapAddScan**() with the same destination result buffer.

> Unlike most other functions that modify an Aurora Imaging Library object, you can call this function concurrently from multiple threads on the same M_POINT_CLOUD_RESULT result buffer (Result3dmapId) without using an M_MUTEX object. This is valid as long as the PointCloudLabel parameter of the concurrent calls is set to a different index.

Parameters

LaserContext3dmapId

Specifies the identifier of the 3D reconstruction context. The 3D reconstruction context must have been previously allocated on the required system using M3dmapAlloc() with M_LASER.

Result3dmapId

Specifies the identifier of the 3D reconstruction result buffer in which to store the laser line data. The 3D reconstruction result buffer must have been previously allocated using M3dmapAllocResult() with

M_LASER_CALIBRATION_DATA, M_POINT_CLOUD_RESULT, or M_DEPTH_CORRECTED_DATA.

> You can call this function concurrently in multiple threads for the same M_POINT_CLOUD_RESULT result buffer, so long as the PointCloudLabel is distinct between concurrent calls and all other parameters do not share data between concurrent calls.

LaserOrDepthMapImageBufId

Specifies the identifier of the image buffer containing the grabbed image of the laser line.

If ControlFlag is set to M_LINE_ALREADY_EXTRACTED, the image buffer must contain an uncorrected depth map that follows the following format:

- If the M_SCAN_LANE_DIRECTION control type is set to M_HORIZONTAL, the uncorrected depth map must contain *M* rows and *N* columns, where:
  - *M* is the number of frames containing the laser line. One frame generates one row of data, where frame *m* corresponds to row *m* in the uncorrected depth map (0 <= *m* <= *M* - 1).
  - *N* is the width, in pixels, of the laser line image. One pixel corresponds to one column of data, where the pixel distance of the laser line in column *n* of the laser line image corresponds to the pixel in column *n* in the uncorrected depth map (0 <= *n* <= *N* - 1).
  - Each entry (*m*, *n*) in the uncorrected depth map must contain a value *d*, which is the vertical distance of the laser line in column *n* of the laser line image *m*.
- If the M_SCAN_LANE_DIRECTION control type is set to M_VERTICAL, the uncorrected depth map must contain *M* rows and *N* columns, where:
  - *M* is the number of frames containing the laser line. One frame generates one row of data, where frame *m* corresponds to row *m* in the uncorrected depth map (0 <= *m* <= *M* - 1).
  - *N* is the height, in pixels, of the laser line image. One pixel corresponds to one row of data, where the pixel distance of the laser line in row *n* of the laser line image corresponds to the pixel in column *n* in the uncorrected depth map (0 <= *n* <= *N* - 1).
  - Each entry (*m*, *n*) in the uncorrected depth map must contain a value *d*, which is the horizontal distance of the laser line at row *n* in the laser line image *m*.

For more information on the required image buffer format in M_LINE_ALREADY_EXTRACTED mode, consult the Using 3D cameras that output uncorrected depth maps section of Chapter 46: 3D reconstruction using laser line profiling.

If you supply **M3dmapAddScan**() with a child buffer so that it processes only a subset of the grabbed laser line image, and you will be generating a fully corrected depth map, you must specify the X- and Y-offsets of that child buffer, relative to the top-left pixel of the image buffer used during camera calibration. To do so, use M3dmapControl() with M_EXTRACTION_CHILD_OFFSET_X and M_EXTRACTION_CHILD_OFFSET_Y. If you know, for example, that the laser line will appear only at certain heights in the grabbed laser line image, using a child buffer can be useful to narrow down the search region and reduce processing time.

This buffer must have been previously allocated with MbufAlloc2d(). The image must be a 1-band, 8-bit or 16-bit, unsigned buffer and its size must not exceed 65535 pixels by 65535 pixels.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

IntensityImageBufId

Specifies the identifier of the image buffer containing the intensity map, if using a 3D camera. If you are not using a 3D camera, set this parameter to **M_NULL**. If you use a 3D camera that does not provide intensity map data (or if

you want to exclude it), you should also set this parameter to **M_NULL**.

If the M_SCAN_LANE_DIRECTION control type of the 3D reconstruction context is set to M_VERTICAL, the gray level of the uncorrected intensity map for coordinate (*X*, *Y*) corresponds to the Y-position of the laser line at column *X*.

If the M_SCAN_LANE_DIRECTION control type of the 3D reconstruction context is set to M_HORIZONTAL, the gray level of the uncorrected intensity map at coordinate (*X*, *Y*) corresponds to the X-position of the laser line at row *Y*.

This image buffer must be a 1-band buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

ExtraInfoArrayId

Reserved for future expansion. This parameter must be set to **M_NULL**.

PointCloudLabel

Specifies the label of the point cloud in the result buffer in which to add the laser line data, if the result buffer has been allocated using M3dmapAllocResult() with M_POINT_CLOUD_RESULT. For all other types of 3D reconstruction result buffers, set this parameter to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the label of a point cloud																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_POINT_CLOUD_LABEL(AIL_INT PointCloudLabel)	Specifies the label of the point cloud. If the result buffer does not contain a point cloud with the specified label, a new point cloud is created in the result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PointCloudLabel INQ Specifies the label.															
	1 <= *Value* <= 134217727 Specifies the label as a positive integer value. This value must be between 1 and 134217727.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies the function's control flag. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To specify the function's control flag																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Same as M_EXTRACT_LINE.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACT_LINE	Extracts the laser line data from the input image, corrects the data if the 3D reconstruction context is calibrated, and stores it in the result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_ALREADY_EXTRACTED	Specifies that the uncorrected depth map and intensity map are given as input. Specify this control flag only if the 3D Reconstruction module should not perform the laser line extraction, as is the case for 3D cameras that perform laser line extraction.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapAlignScan

Synopsis

Calculates the transformations and information necessary to calibrate a 3D profile sensor.

Syntax

```
void M3dmapAlignScan(
    AIL_ID AlignContext3dmapId, //in
    AIL_ID ContainerBufId,      //in
    AIL_ID AlignResult3dmapId,  //out
    AIL_INT64 ControlFlag       //in
)
```

Description

This function calculates the transformations necessary to calibrate your 3D profile sensor using an alignment object that was specified using M3dmapControl() with M_OBJECT_SHAPE.

This function uses a point cloud of an alignment object, with expected markings and constraints, to establish the transformations and information necessary to calibrate the 3D sensor that transmitted the point cloud. The calibration, when applied to the data transmitted by the 3D sensor, corrects the data so that the alignment object is represented correctly.

To retrieve the calculated sensor data, use M3dmapGetResult() and M3dmapCopyResult(). The sensor data includes values such as M_SENSOR_PITCH, M_SENSOR_PITCH_BEFORE_YAW, M_SENSOR_YAW, M_SENSOR_YAW_BEFORE_PITCH, M_3D_SHEAR_X, M_3D_SHEAR_Z, and M_3D_SCALE_Y.

You must use M3dmapControl() with M_OBJECT_SHAPE set to M_PYRAMID, M_DISK, or M_FLAT_SURFACE prior to calling this function. When using an alignment pyramid or disk, certain constraints must be met. You must also specify the dimensions of your alignment object before calling **M3dmapAlignScan**().

- For an alignment disk (M_DISK):
  - You must specify the dimensions of M_HEIGHT and M_DIAMETER using M3dmapControl() before calling **M3dmapAlignScan**().
  - For information on the alignment disk constraints, see Alignment disk constraints and requirements.
- For an alignment pyramid (M_PYRAMID):
  - You must specify the dimensions of M_HEIGHT, M_PYRAMID_BASE_LENGTH, and M_PYRAMID_TOP_FACE_LENGTH using M3dmapControl() before calling **M3dmapAlignScan**().
  - For information on the alignment pyramid constraints, see Alignment pyramid constraints and requirements.

The following are the different ways to calibrate a 3D profile sensor with the calculated sensor data.

- You can use M3dimMatrixTransform() to apply the calculated transformation matrix (M3dmapCopyResult() with M_TRANSFORMATION_MATRIX) to point clouds acquired by the 3D profile sensor.
- You can use MbufConvert3d() when converting the 3D sensor data to a format that is 3D-processable. To do so, prior to calling MbufConvert3d(), set MbufControl() with M_3D_SHEAR_X, M_3D_SCALE_Y, M_3D_SHEAR_Z to the values retrieved using M3dmapGetResult() with M_3D_SHEAR_X, M_3D_SCALE_Y, M_3D_SHEAR_Z, respectively.
- You can pass the calculated results (such as pitch, yaw, and shear values) to a compatible 3D profile sensor (such as Zebra AltiZ) using MdigControlFeature(). In this case, the 3D profile sensor will directly output

calibrated scan lines.

Parameters

AlignContext3dmapId

Specifies the identifier of the 3D alignment context to use for the alignment operation. The context must have been previously allocated using M3dmapAlloc() with M_ALIGN_CONTEXT.

ContainerBufId

Specifies the identifier of the 3D-processable point cloud container that stores the point cloud of the alignment object.

The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

AlignResult3dmapId

Specifies the identifier of the 3D alignment result buffer in which to store results. The result buffer must have been previously allocated using M3dmapAllocResult() with M_ALIGN_RESULT.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapAlloc

Synopsis

Allocate a 3D reconstruction context, 3D draw context, or 3D alignment context.

Syntax

```
AIL_ID M3dmapAlloc(
    AIL_ID SysId,                       //in
    AIL_INT64 ContextType,              //in
    AIL_INT64 ControlFlag,              //in
    AIL_ID *ContextOrGeometry3dmapIdPtr //out
)
```

Description

This function allocates a 3D reconstruction context or 3D alignment context on the specified system. A 3D reconstruction context contains information needed to perform an M3dmapAddScan() operation and a 3D alignment context contains information needed to perform an M3dmapAlignScan() operation. You can also use **M3dmapAlloc**() to allocate a 3D draw context for drawing results using M3dmapDraw3d().

When the 3D reconstruction context, 3D draw context, or 3D alignment context is no longer required, release it using M3dmapFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the 3D reconstruction context, 3D draw context, or 3D alignment context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the identifier of the system on which to allocate the 3D reconstruction context, 3D draw context, or 3D alignment context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextType

Specifies to allocate a 3D reconstruction context, 3D draw context, or 3D alignment context. This parameter must be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_ALIGN_CONTEXT	Specifies a 3D alignment context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_3D_CONTEXT	Specifies a 3D draw context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LASER	Specifies a 3D reconstruction context that will be used to perform laser line (sheet of light) profiling.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag INQ

Specifies the 3D reconstruction mode of the context, which determines how data is represented. The data representation determines whether depth maps can be fully corrected and whether 3D point clouds can be returned.

If allocating a 3D draw context or 3D alignment context, this parameter must be set to **M_DEFAULT**.

If allocating a 3D reconstruction context, this parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 3D reconstruction mode																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CALIBRATED_CAMERA_LINEAR_MOTION +	Specifies that the 3D reconstruction context will include camera calibration information and depth correction information. This allows for the storage of 3D point clouds and the generation of fully corrected depth maps (shape and depth corrected). > Note that the camera calibration context must have been allocated using McalAlloc() with M_TSAI_BASED or M_3D_ROBOTICS.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_CORRECTION	Specifies that the 3D reconstruction context will include depth correction information, but will not include camera calibration information. This allows for only partially corrected depth maps (depth, but not shape corrected).	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_CALIBRATED_CAMERA_LINEAR_MOTION.

You can add one or more of the following values to the above-mentioned value to set the label of the camera or laser.

This is necessary when creating a multiple camera-laser pair setup.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the camera label or laser label for a given context																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_CAMERA_LABEL(AIL_INT CameraLabel)	Specifies the label for the camera used by this 3D reconstruction context. Note that multiple 3D reconstruction contexts can share the same camera, and in those cases, they must also share the same camera label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	CameraLabel INQ Specifies the value of a camera label.															
M_LASER_LABEL(AIL_INT LaserLabel)	Specifies the label for the laser used by this 3D reconstruction context. Note that multiple 3D reconstruction contexts can share the same laser, and in those cases, they must also share the same laser label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LaserLabel INQ Specifies the value of a laser label.															

ContextOrGeometry3dmapIdPtr

Specifies the address of the variable in which to write the 3D reconstruction context, 3D draw context, or 3D alignment context identifier or specifies the data type that the function should use to return the 3D reconstruction context, 3D draw context, 3D alignment context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D reconstruction context, 3D draw context, or 3D alignment context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D reconstruction context, 3D draw context, or 3D alignment context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_3DMAP_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the 3D reconstruction context, 3D draw context, or 3D alignment context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the 3D alignment context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated 3D alignment context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the 3D draw context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated 3D draw context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the 3D reconstruction context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated 3D reconstruction context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the 3D reconstruction context, 3D draw context, or 3D alignment context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_3DMAP_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapAllocResult

Synopsis

Allocate a 3D reconstruction or alignment result buffer.

Syntax

```
AIL_ID M3dmapAllocResult(
    AIL_ID SysId,            //in
    AIL_INT64 ResultType,    //in
    AIL_INT64 ControlFlag,   //in
    AIL_ID *Result3dmapIdPtr //out
)
```

Description

This function allocates a 3D reconstruction or alignment result buffer on the specified system.

When the 3D reconstruction or alignment result buffer is no longer required, release it using M3dmapFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the 3D reconstruction or alignment result buffer's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the identifier of the system on which to allocate the result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result buffer to allocate. This parameter must be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of result buffer to allocate																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALIGN_RESULT	Specifies to allocate a 3D alignment result buffer. This result buffer holds the results produced from calling M3dmapAlignScan() with a 3D alignment context (M_ALIGN_CONTEXT).	c		e	f	g	a		i		j	k	l	m	n	b

M_DEPTH_CORRECTED_DATA	Specifies to allocate a 3D reconstruction result buffer used to store results generated in M_DEPTH_CORRECTION mode, after the 3D reconstruction context has been calibrated.	c		e	f	g	a		i		j	k	l	m	n	b
M_LASER_CALIBRATION_DATA	Specifies to allocate a 3D reconstruction result buffer used to store images of laser line displacement at specified heights during the 3D reconstruction calibration process.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_CLOUD_RESULT	Specifies to allocate a 3D reconstruction result buffer used to store results generated in M_CALIBRATED_CAMERA_LINEAR_MOTION mode, after the 3D reconstruction context has been calibrated.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Result3dmapIdPtr

Specifies the address of the variable in which to write the 3D reconstruction or alignment result buffer identifier or specifies the data type that the function should use to return the 3D reconstruction or alignment result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D reconstruction or alignment result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D reconstruction or alignment result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_3DMAP_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the 3D reconstruction or alignment result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the 3D alignment result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated 3D alignment result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the 3D reconstruction result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated 3D reconstruction result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the 3D reconstruction or alignment result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_3DMAP_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapCalibrate

Synopsis

Calibrate a 3D reconstruction setup.

Syntax

```
void M3dmapCalibrate(
    AIL_ID Context3dmapId, //in
    AIL_ID Result3dmapId,  //in
    AIL_ID ContextCalId,   //in
    AIL_INT64 ControlFlag  //in
)
```

Description

This function calibrates your 3D reconstruction setup.

If the specified 3D reconstruction context is set to generate fully corrected depth maps (M_CALIBRATED_CAMERA_LINEAR_MOTION 3D reconstruction mode), the 3D reconstruction calibration is done using the specified camera calibration context and the extracted laser line data stored in the result buffer. In this mode, the following setup constraints must be met:

- You must specify a 3D-based camera calibration context that was allocated using McalAlloc() with M_TSAI_BASED or M_3D_ROBOTICS.
- If the laser plane is not perpendicular to the object's motion (that is, if it is not perfectly vertical), the result buffer must contain laser line data extracted from multiple reference planes (multiple calls to M3dmapAddScan()). The laser plane is assumed to be perfectly vertical if a single reference plane is used.
- The laser line on the XY-plane (Z=0) must be perfectly perpendicular to the object's motion.
- The camera must have a sufficient angle with respect to the Z-axis to have accurate results; that is, the camera must be able to grab meaningful displacements of the laser line for different depths. For more information, see the Camera angle requirement subsection of the Configuring the laser line profiling setup section of Chapter 46: 3D reconstruction using laser line profiling.

If the specified 3D reconstruction context is set to generate partially corrected depth maps (M_DEPTH_CORRECTION 3D reconstruction mode), the 3D reconstruction calibration is done with the extracted laser line data stored in the result buffer. With this type of context, there is no constraint on the orientation of the laser plane; that is, it does not need to be perfectly vertical or perpendicular to the object's motion. You do not need to provide a camera calibration context and the ContextCalId parameter must be set to **M_NULL**.

For the steps required to calibrate your 3D reconstruction setup in either mode, see the Calibrating your 3D reconstruction setup to create a point cloud section of Chapter 46: 3D reconstruction using laser line profiling.

Parameters

Context3dmapId

Specifies the identifier of the 3D reconstruction context used to extract the laser line from grabbed images. The context must have been previously allocated on the required system using M3dmapAlloc().

Result3dmapId

Specifies the identifier of the 3D reconstruction result buffer that contains the laser line data required for calibration. The 3D reconstruction result buffer must have been previously allocated using M3dmapAllocResult() and contain data for at least one laser line.

ContextCalId

Specifies the identifier of the camera calibration context that contains the 3D camera calibration data or **M_NULL**.

For fully corrected depth maps, this parameter must be set to the identifier of a camera calibration context allocated using McalAlloc() with M_TSAI_BASED or M_3D_ROBOTICS.

For partially corrected depth maps, this parameter must be set to **M_NULL**.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapCalibrateMultiple

Synopsis

Calibrate a 3D reconstruction setup that uses multiple camera-laser pairs.

Syntax

```
void M3dmapCalibrateMultiple(
    const AIL_ID *Context3dmapIdArrayPtr, //in
    const AIL_ID *Result3dmapIdArrayPtr,  //in
    const AIL_ID *ContextCalIdArrayPtr,   //in
    AIL_INT ArraySize,                    //in
    AIL_INT64 ControlFlag                 //in
)
```

Description

This function calibrates your 3D reconstruction setup when using multiple camera-laser pairs.

By identifying the camera-laser pairs using M_CAMERA_LABEL() and M_LASER_LABEL() when allocating the 3D reconstruction context, you can determine which cameras and lasers should share a common 3D reconstruction calibration. When two contexts share the same laser label, they will be constrained to sharing the same laser plane equation. Similarly, when two contexts share the same camera label, they will be constrained to using the same camera calibration. By calling **M3dmapCalibrateMultiple**(), instead of using multiple calls to M3dmapCalibrate(), the 3D reconstruction calibrations will be optimized to satisfy the constraints imposed by all the various calibrations.

When constructing the arrays specified by Context3dmapIdArrayPtr, Result3dmapIdArrayPtr, and ContextCalIdArrayPtr, you must establish a one to one relationship between each item; that is, the 3D reconstruction context is calibrated using the camera calibration and extracted laser line data found in the same array position. The three arrays passed to the function must have the same size specified by ArraySize.

When using **M3dmapCalibrateMultiple**(), the following setup constraints must be met:

- Each camera calibration context must have been allocated using McalAlloc() with M_TSAI_BASED or M_3D_ROBOTICS.
- If the laser planes are not perpendicular to the object's motion (that is, if they are not perfectly vertical), the result buffers must contain laser line data extracted from multiple reference planes (using multiple calls to M3dmapAddScan() for each result buffer). The laser planes are assumed to be perfectly vertical if a single reference plane is provided for a given laser label.
- The laser lines projected on the XY-plane (Z=0) must be perpendicular to the object's motion.
- The camera must have a sufficient angle with respect to their respective laser planes to have accurate results; that is, the cameras must be able to acquire meaningful displacements of the laser lines for different depths. For more information, see the Camera angle requirement subsection of the Configuring the laser line profiling setup section of Chapter 46: 3D reconstruction using laser line profiling.
- The camera calibration identifiers at two distinct indices in the ContextCalIdArrayPtr array must be the same if and only if the camera labels of the two 3D reconstruction contexts at the same indices in the Context3dmapIdArrayPtr array are also the same.

For the steps required to calibrate your 3D reconstruction setup in either mode, see the Calibrating your 3D reconstruction setup to create a point cloud section of Chapter 46: 3D reconstruction using laser line profiling.

After calling **M3dmapCalibrateMultiple**(), you can use M3dmapInquire() set to M_CALIBRATION_STATUS on each 3D reconstruction context to know the results of the 3D reconstruction calibration operation.

- If **M3dmapCalibrateMultiple**() is successful, all statuses are set to M_CALIBRATED.
- If **M3dmapCalibrateMultiple**() fails because of a particular 3D reconstruction context, its calibration status is set according to the cause of failure. An Aurora Imaging Library error is also reported. The 3D reconstruction calibration status of the other contexts are set to M_NOT_INITIALIZED. Apart from the calibration status, the 3D reconstruction contexts are unchanged, and can thus be used in a subsequent call to **M3dmapCalibrateMultiple**().
- If **M3dmapCalibrateMultiple**() fails during the global optimization phase, the calibration status of all the 3D reconstruction contexts is set to the cause of failure. A different Aurora Imaging Library error is also reported. Apart from the calibration status, the 3D reconstruction contexts are unchanged, and can thus be used in a subsequent call to **M3dmapCalibrateMultiple**().
- If **M3dmapCalibrateMultiple**() fails because of an invalid parameter passed to the function, the 3D reconstruction contexts remains unchanged (including the 3D reconstruction calibration status). An Aurora Imaging Library error is reported accordingly.

> This function can only be called on 3D reconstruction contexts allocated with M_CALIBRATED_CAMERA_LINEAR_MOTION reconstruction mode.

Parameters

Context3dmapIdArrayPtr
Accepts the address of one of the following:

- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(ArraySize)

Specifies the address of an array containing the identifiers of the 3D reconstruction context used to extract the laser line from grabbed images. The contexts must have been previously allocated on the required system using M3dmapAlloc().

This array must have a size equal to ArraySize.

Result3dmapIdArrayPtr
Accepts the address of one of the following:

- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(ArraySize)

Specifies the address of an array containing the identifiers of the 3D reconstruction result buffers that contains the laser line data required for calibration. The 3D reconstruction result buffers must have been previously allocated using M3dmapAllocResult() and contain data for at least one laser line.

This array must have a size equal to ArraySize.

ContextCalIdArrayPtr
Accepts the address of one of the following:

- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(ArraySize)

Specifies the address of an array containing the identifiers of the camera calibration contexts of the cameras used to grab the laser lines. The contexts must have been previously allocated on the required system using McalAlloc() with M_TSAI_BASED or M_3D_ROBOTICS and must be calibrated.

This array must have a size equal to ArraySize.

ArraySize

Specifies the number of camera-laser pairs. This parameter must be greater or equal to 1.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the Context3dmapIdArrayPtr, Result3dmapIdArrayPtr, or ContextCalIdArrayPtr parameter.

ControlFlag

Specifies the function's control flag. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying error reporting in the case of duplicate labels																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default behavior. If two or more 3D reconstruction contexts in Context3dmapIdArrayPtr have been allocated with the same values for CameraLabel and LaserLabel, an error is reported.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALLOW_DUPLICATES	Specifies to not report an error if two or more 3D reconstruction contexts in Context3dmapIdArrayPtr have been allocated with the same values for CameraLabel and LaserLabel.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapClear

Synopsis

Clear a 3D reconstruction result buffer of some or all of its content.

Syntax

```
void M3dmapClear(
    AIL_ID Result3dmapId, //out
    AIL_INT LabelOrIndex, //in
    AIL_INT64 Operation,  //in
    AIL_INT64 ControlFlag //in
)
```

Description

This function allows you to clear or delete the laser line data from a 3D reconstruction result buffer without having to free and reallocate the buffer.

For M_POINT_CLOUD_RESULT result buffers, you can clear a point cloud of its points while keeping a record of the ongoing conveyor movement with M_CLEAR. You can also delete a point cloud, which deletes its points, deletes its record of ongoing conveyor movement, and frees its label, with M_DELETE. Note that when you clear or delete a point cloud, the relative coordinate system and some other control value information remain in the M_POINT_CLOUD_RESULT result buffer.

For M_DEPTH_CORRECTED_DATA result buffers, you can clear the entire result buffer of all laser data using M_CLEAR. The result buffer still retains some control value information.

During calibration of the 3D reconstruction setup, you can remove the last scan added to the M_LASER_CALIBRATION_DATA result buffer, for instance because you know the last scan was in error, using M_REMOVE_LAST_SCAN.

> Unlike most other functions that modify an Aurora Imaging Library object, you can call this function concurrently from multiple threads on the same M_POINT_CLOUD_RESULT result buffer (Result3dmapId) without using an M_MUTEX object. This is valid as long as the LabelOrIndex parameter of the concurrent calls is set to a different index, and is not set to M_ALL.

Parameters

Result3dmapId

Specifies the identifier of the 3D reconstruction result buffer to clear. The 3D reconstruction result buffer must have been previously allocated using M3dmapAllocResult() with M_POINT_CLOUD_RESULT, M_LASER_CALIBRATION_DATA, or M_DEPTH_CORRECTED_DATA. Any other type of 3D reconstruction result buffer will cause an error.

LabelOrIndex

Specifies the point cloud(s) in the specified 3D reconstruction result buffer. Only 3D reconstruction result buffers of type M_POINT_CLOUD_RESULT have point clouds that can be specified using this parameter. For all other types of 3D reconstruction result buffers, set this parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a point cloud																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_CLOUD_INDEX(AIL_INT PointCloudIndex)	Specifies the index of a point cloud in the specified 3D reconstruction result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PointCloudIndex Specifies the index of an existing point cloud.															
M_POINT_CLOUD_LABEL(AIL_INT PointCloudLabel)	Specifies the label for a point cloud in the specified 3D reconstruction result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PointCloudLabel Specifies the label of an existing point cloud.															
M_ALL	Specifies all point clouds in the specified 3D reconstruction result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Operation

Specifies the operation to perform. This parameter must be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CLEAR	Clears laser line data from a 3D reconstruction result buffer. For an M_POINT_CLOUD_RESULT result buffer, this clears all the points in the specified point cloud, while maintaining the Y-axis displacement (ongoing conveyor movement) since the first call to M3dmapAddScan(). The relative coordinate system associated with the M_POINT_CLOUD_RESULT result buffer, along with some control values, remain in the result buffer. For an M_DEPTH_CORRECTED_DATA result buffer, this clears all laser line data, although some control values remain. Note that result buffers of type M_DEPTH_CORRECTED_DATA do not have a relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_DELETE	Deletes one (or all) point cloud(s) from the collection of point clouds in an M_POINT_CLOUD_RESULT result buffer. This deletes the points in the point cloud, the Y-axis displacement, and the point cloud label of the point cloud. You can create a new point cloud using this freed label.	c		e	f	g	a		i		j	k	l	m	n	b

M_REMOVE_LAST_SCAN	Discards the most recent scan added to the result buffer. This is available only for M_LASER_CALIBRATION_DATA result buffers. This operation is useful during the 3D reconstruction calibration phase if you know that the quality of the most recent scan added to the result buffer is not good enough for an accurate calibration.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapControl

Synopsis

Control a 3D alignment context, profiling 3D reconstruction context, draw 3D reconstruction context, or 3D reconstruction result buffer.

Syntax

```
void M3dmapControl(
    AIL_ID ContextOrResult3dmapId, //out
    AIL_INT LabelOrIndex,          //in
    AIL_INT64 ControlType,         //in
    AIL_DOUBLE ControlValue        //in
)
```

Description

This function allows you to control a specified setting of a 3D alignment context, profiling 3D reconstruction context, draw 3D reconstruction context, or 3D reconstruction result buffer.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **M3dmapControl()** internally calls the *AIL_DOUBLE* version of this function (M3dmapControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (M3dmapControlInt64()).

Parameters

ContextOrResult3dmapId

Specifies the identifier of the 3D alignment context, profiling 3D reconstruction context, draw 3D reconstruction context, or 3D reconstruction result buffer whose setting to modify. The 3D alignment context, profiling 3D reconstruction context, or draw 3D reconstruction context must have been previously allocated using M3dmapAlloc() with M_ALIGN_CONTEXT, M_LASER, or M_DRAW_3D_CONTEXT, respectively. The 3D reconstruction must have been previously allocated using M3dmapAllocResult() with M_DEPTH_CORRECTED_DATA, M_LASER_CALIBRATION_DATA, or M_POINT_CLOUD_RESULT.

LabelOrIndex

Specifies the point cloud(s) in the specified 3D reconstruction result buffer, or the entire result buffer itself, to control. Only 3D reconstruction result buffers allocated using M_POINT_CLOUD_RESULT have point clouds that can be specified using this parameter. For all other types of 3D reconstruction contexts and result buffers, set this parameter to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the point cloud(s) or point cloud container																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_POINT_CLOUD_INDEX(AIL_INT PointCloudIndex)	Specifies the index of a point cloud in the specified 3D reconstruction result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PointCloudIndex Specifies the index of an existing point cloud.															

M_POINT_CLOUD_LABEL(AIL_INT PointCloudLabel)	Specifies the label of a point cloud in the specified 3D reconstruction result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PointCloudLabel Specifies the label of an existing point cloud.															
M_ALL	Specifies to apply the control to all point clouds in the specified 3D reconstruction result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to control the specified 3D reconstruction result buffer allocated using M_POINT_CLOUD_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the type of setting to control.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the setting.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

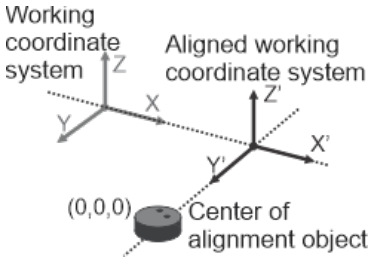
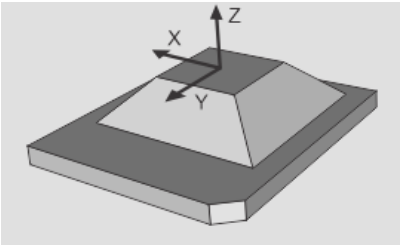
- For a 3D alignment context
- For a profiling 3D reconstruction context
- For a profiling 3D reconstruction context in M_CALIBRATED_CAMERA_LINEAR_MOTION 3D reconstruction mode
- For a 3D reconstruction result buffer set to M_POINT_CLOUD_RESULT, M_DEPTH_CORRECTED_DATA, or M_LASER_CALIBRATION_DATA
- For a 3D reconstruction result buffer set to M_POINT_CLOUD_RESULT
- For a draw 3D reconstruction context

The following ControlType and corresponding ControlValue parameter settings can be specified for a 3D alignment context (M_ALIGN_CONTEXT).

> Measurements that rely on the presence of an alignment object will not be calculated if M_OBJECT_SHAPE is set to M_FLAT_SURFACE. In this case, M_3D_SCALE_Y, M_3D_SHEAR_X, M_3D_SHEAR_Z, M_HOLES_FOUND, M_Y_MIRRORED, M_RMS_ERROR, M_PYRAMID_CORNERS_X, M_PYRAMID_CORNERS_Y, and M_PYRAMID_CORNERS_Z will not be retrievable using M3dmapGetResult().

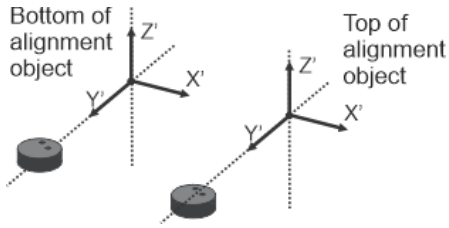
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a 3D alignment context																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																

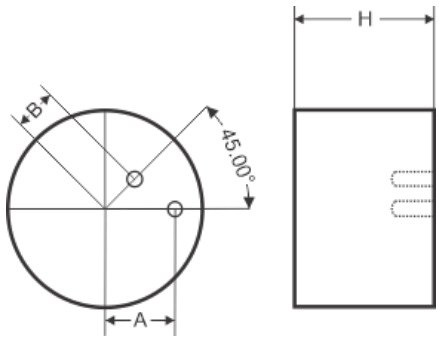
M_ALIGN_X_POSITION		Sets whether to calculate the transformation required to align the origin of the working coordinate system with the origin of the alignment object along the X-axis. The alignment disk's origin is at its center; the alignment pyramid's origin is at the center of its truncated pyramid. INQ  														
	M_DEFAULT	Same as M_SAME.	c		e	f	g	a		i		j	k	l	m	n	b
	M_OBJECT_ORIGIN	Specifies to calculate the transformation to align the origin of the working coordinate system with the origin of the alignment object along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME	Specifies not to calculate the transformation to align the origin of the working coordinate system along the X-axis. The origin of the working coordinate system will remain in its original position.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALIGN_XY_DIRECTION		Sets whether to calculate the transformation required to change the direction of the X- and Y-axes of the working coordinate system.  Note that changing the direction of one axis automatically updates the other axes to maintain the right-hand rule. Also note that Z=0 depends on M_ALIGN_Z_POSITION. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to calculate the transformation to align with default direction; this is specific to the type of alignment object. For the alignment disk (M_DISK) and alignment surface (M_FLAT_SURFACE), specifies to use M_SAME_X. For the alignment pyramid (M_PYRAMID), specifies to use M_OBJECT_SAME_X.	c		e	f	g	a		i		j	k	l	m	n	b

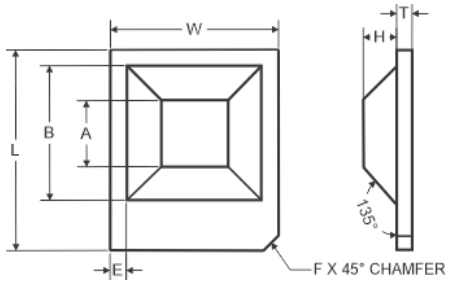
	M_OBJECT_REVERSE_X	Specifies to calculate the transformation to align the X-axis of the working coordinate system with the reverse direction of the X-axis of the alignment pyramid. Depending on the setting of M_ALIGN_Z_DIRECTION, the Y-axis of the working coordinate system will either align with the same or reverse direction of the Y-axis of the alignment pyramid. This control value is only supported for the alignment pyramid (M_PYRAMID).	c		e	f	g	a		i		j	k	l	m	n	b
	M_OBJECT_REVERSE_Y	Specifies to calculate the transformation to align the Y-axis of the working coordinate system with the reverse direction of the Y-axis of the alignment pyramid. Depending on the setting of M_ALIGN_Z_DIRECTION, the X-axis of the working coordinate system will either align with the same or reverse direction of the X-axis of the alignment pyramid. This control value is only supported for the alignment pyramid (M_PYRAMID).	c		e	f	g	a		i		j	k	l	m	n	b
	M_OBJECT_SAME_X	Specifies to calculate the transformation to align the X-axis of the working coordinate system with the direction of the X-axis of the alignment pyramid. Depending on the setting of M_ALIGN_Z_DIRECTION, the Y-axis of the working coordinate system will either align with the same or reverse direction of the Y-axis of the alignment pyramid. This control value is only supported for the alignment pyramid (M_PYRAMID).	c		e	f	g	a		i		j	k	l	m	n	b
	M_OBJECT_SAME_Y	Specifies to calculate the transformation to align the Y-axis of the working coordinate system with the direction of the Y-axis of the alignment pyramid. Depending on the setting of M_ALIGN_Z_DIRECTION, the X-axis of the working coordinate system will either align with the same or reverse direction of the X-axis of the alignment pyramid. This control value is only supported for the alignment pyramid (M_PYRAMID).	c		e	f	g	a		i		j	k	l	m	n	b
	M_REVERSE_X	Specifies to calculate the transformation required to reverse the direction of the X-axis of the working coordinate system. Depending on the setting of M_ALIGN_Z_DIRECTION, the Y-axis of the working coordinate system will either point in the current or reverse direction. This control value is only supported for the alignment disk (M_DISK) and alignment surface (M_FLAT_SURFACE).	c		e	f	g	a		i		j	k	l	m	n	b

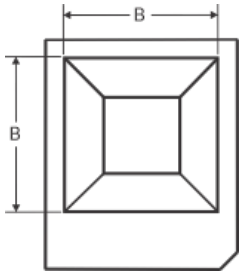
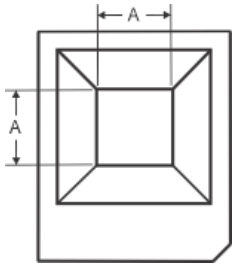
	M_REVERSE_Y	Specifies to calculate the transformation required to reverse the direction of the Y-axis of the working coordinate system. Depending on the setting of M_ALIGN_Z_DIRECTION, the X-axis of the working coordinate system will either point in the current or reverse direction. > This control value is only supported for the alignment disk (M_DISK) and alignment surface (M_FLAT_SURFACE).	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME_X	Specifies to maintain the current direction of the X-axis of the working coordinate system. Depending on the setting of M_ALIGN_Z_DIRECTION, the Y-axis of the working coordinate system will either point in the current or reverse direction. > This control value is only supported for the alignment disk (M_DISK) and alignment surface (M_FLAT_SURFACE).	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME_Y	Specifies to maintain the current direction of the Y-axis of the working coordinate system. Depending on the setting of M_ALIGN_Z_DIRECTION, the X-axis of the working coordinate system will either point in the current or reverse direction. > This control value is only supported for the alignment disk (M_DISK) and alignment surface (M_FLAT_SURFACE).	c		e	f	g	a		i		j	k	l	m	n	b
M_ALIGN_Y_POSITION		Sets whether to calculate the transformation required to align the origin of the working coordinate system with the origin of the alignment object along the Y-axis. The alignment disk's origin is at its center; the alignment pyramid's origin is at the center of its truncated pyramid. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_SAME.	c		e	f	g	a		i		j	k	l	m	n	b
	M_OBJECT_ORIGIN	Specifies to calculate the transformation to align the origin of the working coordinate system with the origin of the alignment object along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME	Specifies not to calculate the transformation to align the origin of the working coordinate system along the Y-axis. The origin of the working coordinate system will remain in its original position.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALIGN_Z_DIRECTION		Sets whether to calculate the transformation required to align the direction of the Z-axis of the working coordinate system. Note that changing the direction of one axis automatically updates the other axes to maintain the right-hand rule. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_SAME.	c		e	f	g	a		i		j	k	l	m	n	b
	M_REVERSE	Specifies to calculate the transformation required to reverse the direction of the Z-axis, relative to its original direction.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME	Specifies not to calculate the transformation required to change the direction of the Z-axis. The current direction of the Z-axis will be maintained.	c		e	f	g	a		i		j	k	l	m	n	b
	M_Z_DOWN	Specifies to calculate the transformation required to point the Z-axis downwards. This means that the Z-coordinates decrease in the direction in which the height increases. > Note that this leads to negative Z-coordinates for the alignment object's surface.	c		e	f	g	a		i		j	k	l	m	n	b
	M_Z_UP	Specifies to calculate the transformation required to point the Z-axis upwards. This means that the Z-coordinates increase in the direction in which the height increase. > Note that this leads to positive Z-coordinates for the alignment object's surface.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALIGN_Z_POSITION		Sets whether to calculate the transformation required to align the origin of the working coordinate system to the top or bottom of the alignment object. INQ  	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_OBJECT_BOTTOM.	c		e	f	g	a		i		j	k	l	m	n	b
	M_OBJECT_BOTTOM	Specifies to calculate the transformation to align the origin of the working coordinate system with the bottom of the alignment object along the Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	M_OBJECT_TOP	Specifies to calculate the transformation to align the origin of the working coordinate system with the top of the alignment object along the Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME	Specifies not to calculate the transformation to align the origin of the working coordinate system along the Z-axis. The origin of the working coordinate system will remain in its original position.	c		e	f	g	a		i		j	k	l	m	n	b

M_CAMERA_TILT_X	Sets whether to enable camera tilt correction around the X-axis, and in which direction to enable the correction. Note that this does not apply to the alignment pyramid (M_PYRAMID) for which the tilt direction is automatically deduced. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ZERO.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEGATIVE	Specifies that the camera tilt is in the negative direction around the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITIVE	Specifies that the camera tilt is in the positive direction around the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_ZERO	Specifies that there is no camera tilt around the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIAMETER	Sets the diameter of the alignment disk. Note that if M_OBJECT_SHAPE is set to M_DISK, you must set M_DIAMETER to a positive value that is not M_UNKNOWN before calling M3dmapAlignScan(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_UNKNOWN.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN	Specifies that the diameter of the alignment disk is unknown.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the diameter of the alignment disk.	c		e	f	g	a		i		j	k	l	m	n	b
M_HEIGHT	Sets the height of the alignment object that was specified with M_OBJECT_SHAPE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the height of the alignment object.	c		e	f	g	a		i		j	k	l	m	n	b
M_OBJECT_SHAPE	Sets the type of alignment object. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISK.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISK	Specifies a disk-type object for alignment (alignment disk). The alignment disk is a short cylinder (which includes two holes at specific offsets with respect to the origin of the alignment disk).  The alignment disk must meet the following constraints: • The alignment disk must cover at least 50% of the scanned width (X-direction). • The alignment disk must cover at least 30% of the scanned length (Y-direction). • The alignment disk's edge must be fully visible in the scan. • The alignment disk's holes must be at least 30 scan lines in the Y-direction and 30 points in the X-direction. The radii of the holes must be within 5 to 10% of the disk's radius, and the depth of the holes must be at least 20% of the total disk's height. • The floor (background) must be present in the scan. If you are using a stage, ensure its surface is parallel to the motion. For more information on the constraints and recommended dimension values, see Alignment cylinder constraints and requirements. > You must define the dimensions of the alignment disk (M_DIAMETER and M_HEIGHT) before calling M3dmapAlignScan().	c		e	f	g	a		i		j	k	l	m	n	b
	M_FLAT_SURFACE	Specifies a flat-surface-type object for alignment (alignment surface). > For example, a flat square-like piece of sandblasted aluminum, or an equivalently flat piece of medium-density fiberboard (MDF).	c		e	f	g	a		i		j	k	l	m	n	b

			c		e	f	g	a		i		j	k	l	m	n	b
	M_PYRAMID	Specifies a pyramid-type object for alignment (alignment pyramid). The alignment pyramid is a truncated square pyramid on a wider rectangular base that has a chamfer in the bottom right corner. The truncated pyramid is off-center along the length on the wider rectangular base.  The alignment pyramid must meet the following constraints: • The top face and base of its truncated pyramid must be squares. • The angle between the top and side faces of its truncated pyramid must be between 35 to 55 degrees (ideally 45 degrees). • Its truncated pyramid's top face length **(A)** must be approximately 50% of the truncated pyramid's bottom base length **(B)**. • The alignment pyramid's wider rectangular base must be approximately 25% wider **(W)** in the X-direction and 50% longer **(L)** in the Y-direction than its truncated pyramid's base **(B)**. • The alignment pyramid must cover at least 50% of the scanned width (X-direction). • The alignment pyramid must cover at least 30% of the scanned length (Y-direction). • Its truncated pyramid's faces (top and sides) and the top of the alignment pyramid's wider rectangular base must each be composed of at least 2000 points. • The alignment pyramid's wider rectangular base must have a chamfer in the bottom right corner **(F)** with a length of at least 30% of the truncated pyramid's top face **(A)** at an approximate angle of 45 degrees. For more information on the constraints and recommended dimension values, see Alignment pyramid constraints and requirements. > You must define the dimensions of the alignment pyramid (M_PYRAMID_BASE_LENGTH, M_PYRAMID_TOP_FACE_LENGTH, and M_HEIGHT) before calling M3dmapAlignScan().															

M_PYRAMID_BASE_LENGTH		Sets the length of the truncated pyramid's bottom base when using an alignment pyramid.  Note that if M_OBJECT_SHAPE is set to M_PYRAMID, you must set M_PYRAMID_BASE_LENGTH to a positive value that is not M_UNKNOWN before calling M3dmapAlignScan(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_UNKNOWN.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNKNOWN	Specifies that the length of the truncated pyramid's bottom base is unknown.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the length of the truncated pyramid's bottom base.	c		e	f	g	a		i		j	k	l	m	n	b
M_PYRAMID_TOP_FACE_LENGTH		Sets the length of the truncated pyramid's top base (top face) when using an alignment pyramid.  Note that if M_OBJECT_SHAPE is set to M_PYRAMID, you must set M_PYRAMID_TOP_FACE_LENGTH to a positive value that is not M_UNKNOWN before calling M3dmapAlignScan(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_UNKNOWN.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNKNOWN	Specifies that the length of the truncated pyramid's top base is unknown.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the length of the truncated pyramid's top base.	c		e	f	g	a		i		j	k	l	m	n	b

M_STEP_LENGTH	Sets the step length (scan step). The step length is the distance between the scan lines. Note that M_STEP_LENGTH has no affect unless M_STEP_LENGTH_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the step length value.	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_LENGTH_MODE	Sets the step length (scan step) mode. The step length is the distance between the scan lines. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_UNKNOWN.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SIZE_Y	Specifies that the step length is provided with the container that stores the point cloud of the alignment object. The M_3D_SCALE_Y result is constrained by the value set using M_STEP_LENGTH.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN	Specifies that the step length is unknown. The M_STEP_LENGTH and M_3D_SCALE_Y are found by analyzing the point cloud without any constraints.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies that the step length is explicitly set with M_STEP_LENGTH. The M_3D_SCALE_Y result is constrained by the value set using M_STEP_LENGTH.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue settings are available for any kind of profiling (M_LASER) 3D reconstruction context:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a profiling 3D reconstruction context																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																

M_CORRECTED_DEPTH		Specifies the Z-coordinate in world units (for point clouds) or gray level (for partially corrected depth maps) used to represent the height of the next reference plane (the next call to M3dmapAddScan()), during 3D reconstruction calibration for depth. If ContextOrResult3dmapId is a profiling 3D reconstruction context allocated with M_CALIBRATED_CAMERA_LINEAR_MOTION, this value must be specified in world units. The specified Z-coordinate is the actual height of the reference plane in the next calibration image. If ContextOrResult3dmapId is a profiling 3D reconstruction context allocated with M_DEPTH_CORRECTION, this value must be specified in gray levels. The specified gray level is paired with the position of the laser line over a reference plane in the calibration image. Note that a partially corrected depth map has no coordinate system, and so there is no actual height involved. INQ														
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *GrayValue* <= 254	Specifies the gray level that will be used to represent the height of the next reference plane, when M3dmapAlloc() is set to M_DEPTH_CORRECTION. For 8-bit depth maps, the range is 0 to 254. Note that 255 is used to indicate an invalid value (no data).	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *GrayValue* <= 65534	Specifies the gray level that will be used to represent the height of the next reference plane, when M3dmapAlloc() is set to M_DEPTH_CORRECTION. For 16-bit depth maps, the range is 0 to 65534. Note that 65535 is used to indicate an invalid value (no data).	c		e	f	g	a		i		j	k	l	m	n	b
	ZCoordinateValue	Specifies the Z-coordinate (in world units) of the next reference plane. Generally, this value will be negative, since the Z-axis typically points downwards and has its origin on the conveyor.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACTION_FIXED_POINT		Sets the number of binary digits used for the fractional part of the gray level values in uncorrected depth maps, when using M3dmapAddScan() with M_LINE_ALREADY_EXTRACTED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 7	Specifies the number of binary digits used for the fractional part of gray level values.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and ControlValue settings are available only for a profiling 3D reconstruction context in M_CALIBRATED_CAMERA_LINEAR_MOTION 3D reconstruction mode:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a profiling 3D reconstruction context in M_CALIBRATED_CAMERA_LINEAR_MOTION 3D reconstruction mode																			
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_EXTRACTION_CHILD_OFFSET_X		Sets the X-offset that M3dmapAddScan() assumes the laser line image buffer to have relative to the top-left pixel of the image buffer used during camera calibration. If you supply M3dmapAddScan() with a child buffer so that it processes only a subset of the grabbed laser line image, and you are generating a fully corrected depth map, you must specify the X- and Y-offsets of that child buffer, relative to the top-left pixel of the image buffer used during camera calibration. If you know, for example, that the laser line will appear only at certain heights in the grabbed laser line image, using a child buffer can be useful to narrow down the search region and reduce processing time. INQ															
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the X-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACTION_CHILD_OFFSET_Y		Sets the Y-offset that M3dmapAddScan() assumes the laser line image buffer to have relative to the top-left pixel of the image buffer used during camera calibration. If you supply M3dmapAddScan() with a child buffer so that it processes only a subset of the grabbed laser line image, and you are generating a fully corrected depth map, you must specify the X- and Y-offsets of that child buffer, relative to the top-left pixel of the image buffer used during camera calibration. If you know, for example, that the laser line will appear only at certain heights in the grabbed laser line image, using a child buffer can be useful to narrow down the search region and reduce processing time. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the Y-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACTION_RANGE_Z		Sets the mode that helps determine the range of valid Z-coordinates for extracted points when using M3dmapAddScan(). An extracted point that has a Z-coordinate outside of this range is set to **M_INVALID_POINT**. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_GREATER	Specifies that the Z-axis range is defined by a lower limit corresponding to M_EXTRACTION_RANGE_Z_LIMIT1. All points having a Z coordinate greater than M_EXTRACTION_RANGE_Z_LIMIT1 are kept.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IN_RANGE	Specifies that the Z-axis range is defined by the inside range of a lower limit and a upper limit corresponding to M_EXTRACTION_RANGE_Z_LIMIT1 and M_EXTRACTION_RANGE_Z_LIMIT2. All points having a Z coordinate greater than or equal to the lower limit and less than or equal to the upper limit are kept. M_EXTRACTION_RANGE_Z_LIMIT1 is the lower limit of the Z-coordinate extraction range when it is lower than M_EXTRACTION_RANGE_Z_LIMIT2. M_EXTRACTION_RANGE_Z_LIMIT1 is the upper limit of the Z-coordinate extraction range when it is higher than M_EXTRACTION_RANGE_Z_LIMIT2.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies that the range covers the entire Z-axis, so all points are kept. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LESS	Specifies that the Z-axis range is defined by a upper limit corresponding to M_EXTRACTION_RANGE_Z_LIMIT1. All points having a Z coordinate less than M_EXTRACTION_RANGE_Z_LIMIT1 are kept.	c		e	f	g	a		i		j	k	l	m	n	b
	M_OUT_RANGE	Specifies that the Z-axis range is defined by the outside range of a lower limit and a upper limit corresponding to M_EXTRACTION_RANGE_Z_LIMIT1 and M_EXTRACTION_RANGE_Z_LIMIT2. M_EXTRACTION_RANGE_Z_LIMIT1 is the lower limit of the Z-coordinate extraction range when it is lower than M_EXTRACTION_RANGE_Z_LIMIT2. M_EXTRACTION_RANGE_Z_LIMIT1 is the upper limit of the Z-coordinate extraction range when it is higher than M_EXTRACTION_RANGE_Z_LIMIT2.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACTION_RANGE_Z_LIMIT1		Sets the first limit value that determines the range of valid Z-coordinates for extracted points. This value is ignored when M_EXTRACTION_RANGE_Z is set to M_INFINITE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the first limit value, which can be either the lower limit or upper limit. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTRACTION_RANGE_Z_LIMIT2	Sets the second limit value that determines the range of valid Z-coordinates for extracted points. Note that when M_EXTRACTION_RANGE_Z is set to M_INFINITE, M_GREATER, or M_LESS, this value is ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the second limit value, which can be either the lower limit or upper limit. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_SPEED	Sets the speed and direction of the object being scanned, along the object's plane of motion. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the speed, in world units per frame. Note that if the object is moving in the negative Y-axis, specify a negative value.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and ControlValue settings are available only for a 3D reconstruction result buffer set to M_POINT_CLOUD_RESULT, M_DEPTH_CORRECTED_DATA, or M_LASER_CALIBRATION_DATA:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a 3D reconstruction result buffer set to M_POINT_CLOUD_RESULT, M_DEPTH_CORRECTED_DATA, or M_LASER_CALIBRATION_DATA																
ControlType ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MAX_FRAMES	Sets the maximum number of scanned laser lines that the 3D reconstruction result buffer should keep internally. Note that, if the value of this control type is changed after it has already been set, laser lines can be lost. When you specify a 3D reconstruction result buffer of type M_POINT_CLOUD_RESULT with LabelOrIndex set to M_GENERAL), this control will set the maximum number of scanned laser lines kept by each subsequent point cloud generated in this result buffer. INQ															
M_DEFAULT	Specifies the default value; the default value is 1024.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum number of scanned laser lines to keep.	c		e	f	g	a		i		j	k	l	m	n	b

The following controls apply to M_POINT_CLOUD_RESULT 3D reconstruction result buffers, when LabelOrIndex is set to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a 3D reconstruction result buffer set to M_POINT_CLOUD_RESULT																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	ControlValue															
M_RESULTS_DISPLACEMENT_MODE		Sets the displacement mode, which determines whether the coordinates of scanned objects include the movement (displacement) of the conveyor. By default, the coordinates of scanned objects are stored as though the objects are fixed at their initial position when the first call to M3dmapAddScan() was made, despite the ongoing movement of the conveyor. When the coordinates are used (for instance, when returning the coordinates or generating a depth map), the displacement mode specifies whether to include the distance that the conveyor moved since the first call to M3dmapAddScan(). When you include this distance, you are getting the real-time coordinates of the objects at the moment the coordinates are returned or the depth map is generated. You can retrieve this conveyor displacement (Y-axis displacement) using M3dmapGetResult() with M_TOTAL_DISPLACEMENT_Y. In some circumstances, such as when manually defining the extraction box when generating a depth map, you might have to keep in mind whether an object's coordinates will include displacement when used. For more information, see the Results displacement mode subsection of the 3D coordinate systems and the coordinates of a point cloud section of Chapter 46: 3D reconstruction using laser line profiling. INQ														
	M_DEFAULT	Same as M_FIXED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CURRENT	Specifies that the results include the ongoing Y-axis displacement. The results return the current position of the object on the conveyor. Specifically, the results will reflect the 3D coordinates at the last call of M3dmapAddScan().	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIXED	Specifies that the results do not include the ongoing Y-axis displacement. The resulting coordinates are those of the object as if it were still at the original position on the conveyor, just prior to being scanned. In this mode, you can use M_RESULTS_DISPLACEMENT_Y to add a displacement offset to the 3D coordinates.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULTS_DISPLACEMENT_Y		Sets the Y-axis displacement to add to the resulting 3D coordinates of an object when M_RESULTS_DISPLACEMENT_MODE is set to M_FIXED. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-axis displacement. > Note that total displacement can be negative depending on the direction of the conveyor.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue settings are available for a draw 3D reconstruction context:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a draw 3D reconstruction context																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_DRAW_ABSOLUTE_COORDINATE_SYSTEM		Sets whether to draw the absolute coordinate system's axes. INQ															
	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to draw the absolute coordinate system's axes.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to draw the absolute coordinate system's axes.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_CAMERA_COORDINATE_SYSTEM		Sets whether to draw the camera coordinate system's axes. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to draw the camera coordinate system's axes.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to draw the camera coordinate system's axes.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_CAMERA_COORDINATE_SYSTEM_NAME		Sets the name to draw for the camera coordinate system; the initial value is "Camera". INQ	c		e	f	g	a		i		j	k	l	m	n	b
	"*String*"	Specifies the name of the camera coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_COORDINATE_SYSTEM_LENGTH	Sets the length at which to draw the specified coordinate system's axes. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the length (in world units) at which to draw the axes of the specified coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_FRUSTUM	Sets whether to draw the frustum of the camera's view. The frustum is the truncated pyramid of vision that originates at the camera's position and, when drawn, ends at the relative XY plane. When enabled, M_DRAW_FRUSTUM draws a wireframe frustum, using the color specified with M_DRAW_FRUSTUM_COLOR. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to draw the frustum.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to draw the frustum.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_FRUSTUM_COLOR	Sets the frustum's color. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COLOR_YELLOW.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies an RGB value to draw in an 8-bit, 3-band buffer. The red, green, and blue values must be values between 0 and 255, inclusive.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Red Specifies the red band, as a value between 0 and 255.															
	Green Specifies the green band, as a value between 0 and 255.															
	Blue Specifies the blue band, as a value between 0 and 255.															

	M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_COLOR	Specifies no color.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_LASER_LINE_COORDINATE_SYSTEM	Sets whether to draw the laser line coordinate system's axes. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to draw the laser line coordinate system's axes.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to draw the laser line coordinate system's axes.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_LASER_PLANE	Sets whether to draw the laser plane. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to draw the laser plane.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to draw the laser plane.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_LASER_PLANE_COLOR_FILL	Sets the laser plane's fill color. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO_COLOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_COLOR	Specifies either the color red or the texture image. If a texture image is specified (using M3dmapDraw3d() with LaserPlaneTextureImageBufId), the texture image is drawn on the laser plane. Otherwise, the plane is drawn with M_COLOR_RED. The texture image is typically the grabbed image of the laser line.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_COLOR	Specifies no color.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TEXTURE_IMAGE	Specifies to use the image passed to M3dmapDraw3d() with LaserPlaneTextureImageBufId, when drawing the laser plane. The texture image is typically the grabbed image of the laser line. Setting M_DRAW_LASER_PLANE_COLOR_FILL to M_TEXTURE_IMAGE and then calling M3dmapDraw3d() without passing an image to the LaserPlaneTextureImageBufId parameter causes an error.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_LASER_PLANE_COLOR_OUTLINE		Sets the laser plane's outline color. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_COLOR_WHITE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_COLOR	Specifies no color.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_LASER_PLANE_OPACITY		Sets the laser plane's opacity. This control type is only available when the laser plane's fill color is set to a uniform color. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 20.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the laser plane's opacity.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void M3dmapControlInt64 (AIL_ID ContextOrResult3dmapId, AIL_INT LabelOrIndex, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

ContextOrResult3dmapId

See ContextOrResult3dmapId of the main function for a description.

LabelOrIndex

See LabelOrIndex of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void M3dmapControlDouble (AIL_ID ContextOrResult3dmapId, AIL_INT LabelOrIndex, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ContextOrResult3dmapId

See ContextOrResult3dmapId of the main function for a description.

LabelOrIndex

See LabelOrIndex of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapCopy

Synopsis

Copy one or more point clouds from one M_POINT_CLOUD_RESULT result buffer to another.

Syntax

```
void M3dmapCopy(
    AIL_ID SrcResult3dmapId, //in
    AIL_INT SrcLabelOrIndex, //in
    AIL_ID DstResult3dmapId, //out
    AIL_INT DstLabel,        //in
    AIL_INT64 Operation,     //in
    AIL_INT64 ControlFlag    //in
)
```

Description

This function copies or moves one or more point clouds from a source M_POINT_CLOUD_RESULT result buffer to a destination M_POINT_CLOUD_RESULT result buffer.

When copying/moving a specific point cloud, you must specify the label to assign it in the destination result buffer (using DstLabel); the label cannot already be assigned to an existing point cloud in the destination result buffer. If copying/moving all point clouds in the source result buffer (M_ALL), the destination result buffer must be empty; the copied/moved point clouds retain their original labels.

Parameters

SrcResult3dmapId

Specifies the identifier of the 3D reconstruction result buffer containing the source point clouds to copy/move. The 3D reconstruction result buffer must have been previously allocated using M3dmapAllocResult() with M_POINT_CLOUD_RESULT.

SrcLabelOrIndex

Specifies the point cloud(s) to copy/move from the source M_POINT_CLOUD_RESULT result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the label or index of a specific source point cloud

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_CLOUD_INDEX(AIL_INT PointCloudIndex)	Specifies an existing point cloud with the given index. **Parameters** PointCloudIndex Specifies the index of an existing point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_CLOUD_LABEL(AIL_INT PointCloudLabel)	Specifies an existing point cloud with the given label.	c		e	f	g	a		i		j	k	l	m	n	b

	Parameters														
	PointCloudLabel Specifies the label of an existing point cloud.														
M_ALL	Specifies all point clouds in the source M_POINT_CLOUD_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

DstResult3dmapId

Specifies the identifier of the 3D reconstruction result buffer in which to copy/move the source point cloud(s). The 3D reconstruction result buffer must have been previously allocated using M3dmapAllocResult() with M_POINT_CLOUD_RESULT. You can specify the same identifier as the SrcResult3dmapId identifier, but the label to assign the copied/moved point cloud must be different.

DstLabel

Specifies a new label in the destination M_POINT_CLOUD_RESULT result buffer in which to copy/move the source point cloud(s).

If SrcLabelOrIndex is set to M_ALL and the source result buffer contains more than one point cloud, the DstLabel parameter also must be set to M_ALL. In this case, the copied/moved point clouds retain their original source labeling.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the label in the destination result buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_CLOUD_LABEL(AIL_INT PointCloudLabel)	Specifies the label to assign the specific point cloud in the destination buffer. This point cloud should not already exist in the destination result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PointCloudLabel Specifies the label for a point cloud.															
M_ALL	Specifies all labels in the destination M_POINT_CLOUD_RESULT result buffer, which should not contain any point clouds.	c		e	f	g	a		i		j	k	l	m	n	b

Operation

Specifies the type of operation to perform. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_COPY +	Specifies the point cloud is copied to the destination M_POINT_CLOUD_RESULT result buffer; the point cloud is not deleted from the source M_POINT_CLOUD_RESULT result buffer. > Note that the specified point cloud retains its organizational type when copied, unless only valid points are copied (M_EXCLUDE_INVALID_POINTS).	c		e	f	g	a		i		j	k	l	m	n	b
M_MOVE	Specifies the point cloud is moved to the destination M_POINT_CLOUD_RESULT result buffer; the point cloud is deleted from the source M_POINT_CLOUD_RESULT result buffer. This operation is faster than copying. Any point cloud moved using **M3dmapCopy**() retains its organizational type.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_COPY.

You can add the following value to the above-mentioned value to specify to exclude invalid points.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to exclude invalid points																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EXCLUDE_INVALID_POINTS	Specifies to exclude all points in the specified M_POINT_CLOUD_RESULT result buffer that are set to **M_INVALID_POINT**. Only the valid points will be copied.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapCopyResult

Synopsis

Copy a group of results from a 3D alignment result buffer or 3D reconstruction result buffer into a container, an image buffer, or a 3D transformation matrix.

Syntax

```
void M3dmapCopyResult(
    AIL_ID SrcResult3dmapId, //in
    AIL_INT SrcLabelOrIndex, //in
    AIL_ID DstAilObjectId,   //out
    AIL_INT64 CopyType,      //in
    AIL_INT64 ControlFlag    //in
)
```

Description

This function copies a group of results (for example, a point cloud) from a 3D reconstruction result buffer into a container or an image buffer. This function also copies calibration and alignment parameters from a 3D alignment result buffer into a 3D transformation matrix.

Parameters

SrcResult3dmapId

Specifies the identifier of the 3D reconstruction result buffer containing point cloud or laser line data, or the 3D alignment result buffer containing calibration and alignment parameters. The 3D alignment result buffer must have been previously allocated using M3dmapAllocResult() with M_ALIGN_RESULT. The 3D reconstruction result buffer must have been previously allocated using M3dmapAllocResult() with M_DEPTH_CORRECTED_DATA, M_LASER_CALIBRATION_DATA, or M_POINT_CLOUD_RESULT.

See the Parameter associations section for possible values that can be specified.

SrcLabelOrIndex

Specifies the point cloud(s) in the specified 3D reconstruction result buffer to copy. Only 3D reconstruction result buffers allocated using M_POINT_CLOUD_RESULT have point clouds. For all other 3D reconstruction result buffer types, set this parameter to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the point cloud(s)																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_POINT_CLOUD_INDEX(AIL_INT PointCloudIndex)	Specifies an existing point cloud with the given index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PointCloudIndex Specifies the index of an existing point cloud.															
M_POINT_CLOUD_LABEL(AIL_INT PointCloudLabel)	Specifies an existing point cloud with the given label.	c		e	f	g	a		i		j	k	l	m	n	b

	Parameters														
	PointCloudLabel Specifies the label of an existing point cloud.														
M_ALL	Specifies all point clouds in the specified 3D reconstruction result buffer. > M_ALL is supported only when CopyType is set to M_POINT_CLOUD_UNORGANIZED. > Note that the destination container can hold only 1 point cloud. If M_ALL is specified, the point clouds are merged into one.	c		e	f	g	a		i		j	k	l	m	n	b

DstAilObjectId

Specifies the identifier of the destination container, image buffer, or transformation matrix in which to save the copied information.

See the Parameter associations section for possible values that can be returned.

CopyType

Specifies the type of copy operation to perform.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Specifies whether to copy reflectance values, when the SrcResult3dmapId parameter is set to an M_POINT_CLOUD_RESULT result buffer. If the source is not an M_POINT_CLOUD_RESULT result buffer, set this parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the control flag																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to copy intensity values to the destination container's M_COMPONENT_REFLECTANCE component, which is created if not previously existing in the destination container. Note that, if no intensity values are available in the source point cloud, then no reflectance component is created.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_REFLECTANCE	Specifies not to copy intensity values.	c		e	f	g	a		i		j	k	l	m	n	b

Parameter associations

The table below lists possible values for the **SrcResult3dmapId** and **CopyType** parameters and possible values returned to the **DstAilObjectId** parameter.

- For specifying the copy type and destination object for a source 3D reconstruction result buffer or 3D alignment result buffer

> Note that these copy types are only available after a successfully completed alignment operation (when

> M_STATUS is M_COMPLETE) or if a status error is returned (when M_STATUS is M_RESULTS_Z_NEED_BACKGROUND, M_MAYBE_IMPRECISE, or M_CHAMFER_NOT_FOUND).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the copy type and destination object for a source 3D reconstruction result buffer or 3D alignment result buffer

SrcResult3dmapId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
CopyType																
DstAilObjectId - Possible values returned																
3D alignment result buffer identifier	Specifies the identifier of a 3D alignment result buffer (M_ALIGN_RESULT) that stores results generated using M3dmapAlignScan().															
M_RIGID_MATRIX	Specifies to copy the full rigid transformation matrix that aligns the working coordinate system with the alignment object's coordinate system, depending on the settings of M3dmapControl() with M_ALIGN_XY_DIRECTION, M_ALIGN_Z_DIRECTION, M_ALIGN_X_POSITION, M_ALIGN_Y_POSITION, and M_ALIGN_Z_POSITION.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object identifier	Specifies the identifier of the transformation matrix object in which to copy the alignment matrix. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_MATRIX	Specifies to copy the full rotation transformation matrix that aligns the working coordinate system with the alignment object's coordinate system, depending on the settings of M3dmapControl() with M_ALIGN_XY_DIRECTION, and M_ALIGN_Z_DIRECTION.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object identifier	Specifies the identifier of the transformation matrix object in which to copy the alignment matrix. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
M_SHEAR_MATRIX	Specifies to copy the transformation matrix that corrects the shear and scale.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object identifier	Specifies the identifier of the transformation matrix object in which to copy the alignment matrix. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

	M_TRANSFORMATION_MATRIX	Specifies to copy the full transformation matrix that corrects the shear and scale, and aligns the working coordinate system with the alignment object's coordinate system, depending on the settings of M3dmapControl() with M_ALIGN_XY_DIRECTION, M_ALIGN_Z_DIRECTION, M_ALIGN_X_POSITION, M_ALIGN_Y_POSITION, and M_ALIGN_Z_POSITION.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object identifier	Specifies the identifier of the transformation matrix object in which to copy the alignment matrix. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
	M_XY_RIGID_MATRIX	Specifies to copy the rigid transformation matrix that aligns the working coordinate system with the alignment object's coordinate system, depending on the settings of M3dmapControl() with M_ALIGN_Z_DIRECTION, M_ALIGN_X_POSITION, M_ALIGN_Y_POSITION, and M_ALIGN_Z_POSITION. The transformation matrix enforces the Z-axis to be perpendicular to the floor or conveyor, with no enforcement on the X- and Y-axes. Although the X- and/or Y-axes might move, the transformation to align the X- and Y-axes with that of the alignment object is not included. Note that for the alignment disk, this is equivalent to M_RIGID_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object identifier	Specifies the identifier of the transformation matrix object in which to copy the alignment matrix. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
	M_XY_ROTATION_MATRIX	Specifies to copy the rotation transformation matrix that aligns the working coordinate system with the alignment object's coordinate system, depending on the setting of M3dmapControl() with M_ALIGN_Z_DIRECTION. The transformation matrix enforces the Z-axis to be perpendicular to the floor or conveyor, with no enforcement on the X- and Y-axes. Although the X- and/or Y-axes might move, the transformation to align the X- and Y-axes with that of the alignment object is not included. Note that for the alignment disk, this is equivalent to M_ROTATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object identifier	Specifies the identifier of the transformation matrix object in which to copy the alignment matrix. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

M_XY_TRANSFORMATION_MATRIX	Specifies to copy the transformation matrix that corrects the shear and scale, and aligns the working coordinate system with the alignment object's coordinate system, depending on the settings of M3dmapControl() with M_ALIGN_Z_DIRECTION, M_ALIGN_X_POSITION, M_ALIGN_Y_POSITION, and M_ALIGN_Z_POSITION. The transformation matrix enforces the Z-axis to be perpendicular to the floor or conveyor, with no enforcement on the X- and Y-axes. Although the X- and/or Y-axes might move, the transformation to align the X- and Y-axes with that of the alignment object is not included. Note that for the alignment disk, this is equivalent to M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object identifier	Specifies the identifier of the transformation matrix object in which to copy the alignment matrix. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
3D reconstruction depth corrected data result buffer identifier	Specifies the identifier of a 3D reconstruction result buffer that stores results generated in M_DEPTH_CORRECTION mode.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTENSITY_MAP	Copies the intensity map generated using acquired laser line data. In an intensity map, the gray value of each pixel represents the luminous intensity of the laser line at this point. The size of the intensity map corresponds to the size of the uncorrected depth map obtained with M_UNCORRECTED_DEPTH_MAP.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the destination image buffer identifier. The destination image buffer must be a 1-band, 8-bit or 16-bit unsigned buffer. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. The X-size of the destination image buffer should be at least either the X-size or Y-size of the laser line images used to generate the data, depending on whether MimControl() with M_SCAN_LANE_DIRECTION is set to M_VERTICAL or M_HORIZONTAL, respectively. The Y-size of the destination image buffer must be at least as large as the number of scanned laser lines to accumulate in the buffer; this number's maximum possible value is set using M3dmapControl() with M_MAX_FRAMES. Note that the number of scanned laser lines actually accumulated can be lower than this maximum. Use M3dmapGetResult() with M_UNCORRECTED_DEPTH_MAP_SIZE_X, M_UNCORRECTED_DEPTH_MAP_SIZE_Y, and M_INTENSITY_MAP_BUFFER_TYPE to obtain the necessary buffer dimensions, data type, and depth.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PARTIALLY_CORRECTED_DEPTH_MAP	Generates a partially corrected depth map. In a partially corrected depth map, the gray value of a pixel accurately represents real world depth, but any shape distortion (due to the camera's angle) is not corrected.	c		e	f	g	a		i		j	k	l	m	n	b
	Image buffer identifier	Specifies the destination image buffer identifier. The destination image buffer must be a 1-band, 8-bit or 16-bit unsigned buffer. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. The X-size of the destination image buffer should be at least either the X-size or Y-size of the laser line images used to generate the data, depending on whether MimControl() with M_SCAN_LANE_DIRECTION is set to M_VERTICAL or M_HORIZONTAL, respectively. The Y-size of the destination image buffer must be at least as large as the number of scanned laser lines to accumulate in the buffer; this number's maximum possible value is set using M3dmapControl() with M_MAX_FRAMES. Note that the number of scanned laser lines actually accumulated can be lower than this maximum. Use M3dmapGetResult() with M_PARTIALLY_CORRECTED_DEPTH_MAP_SIZE_X, M_PARTIALLY_CORRECTED_DEPTH_MAP_SIZE_Y, and M_PARTIALLY_CORRECTED_DEPTH_MAP_BUFFER_TYPE to obtain the necessary buffer dimensions, data type, and depth.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNCORRECTED_DEPTH_MAP	Copies the uncorrected depth map generated using acquired laser line data. In uncorrected depth maps, the gray value of a pixel is not scaled to the real depth in the world, and any shape distortion (due to the camera's angle) is not corrected.	c		e	f	g	a		i		j	k	l	m	n	b

	Image buffer identifier	Specifies the destination image buffer identifier. The destination image buffer must be a 1-band, 8-bit or 16-bit unsigned buffer. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. The X-size of the destination image buffer should be at least either the X-size or Y-size of the laser line images used to generate the data, depending on whether MimControl() with M_SCAN_LANE_DIRECTION is set to M_VERTICAL or M_HORIZONTAL, respectively. The Y-size of the destination image buffer must be at least as large as the number of scanned laser lines to accumulate in the buffer; this number's maximum possible value is set using M3dmapControl() with M_MAX_FRAMES. Note that the number of scanned laser lines actually accumulated can be lower than this maximum. Use M3dmapGetResult() with M_UNCORRECTED_DEPTH_MAP_SIZE_X, M_UNCORRECTED_DEPTH_MAP_SIZE_Y, and M_UNCORRECTED_DEPTH_MAP_BUFFER_TYPE to obtain the necessary buffer dimensions, data type, and depth. Use M_UNCORRECTED_DEPTH_MAP_FIXED_POINT to obtain the fixed point used to encode the laser line Y position.	c		e	f	g	a		i		j	k	l	m	n	b
3D reconstruction laser calibration result buffer identifier		Specifies the identifier of a 3D reconstruction result buffer that stores images of laser line displacement at specified heights during the 3D reconstruction calibration process.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTENSITY_MAP	Copies the intensity map generated using acquired laser line data. In an intensity map, the gray value of each pixel represents the luminous intensity of the laser line at this point. The size of the intensity map corresponds to the size of the uncorrected depth map obtained with M_UNCORRECTED_DEPTH_MAP.	c		e	f	g	a		i		j	k	l	m	n	b

	Image buffer identifier	Specifies the destination image buffer identifier. The destination image buffer must be a 1-band, 8-bit or 16-bit unsigned buffer. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. The X-size of the destination image buffer should be at least either the X-size or Y-size of the laser line images used to generate the data, depending on whether MimControl() with M_SCAN_LANE_DIRECTION is set to M_VERTICAL or M_HORIZONTAL, respectively. The Y-size of the destination image buffer must be at least as large as the number of scanned laser lines to accumulate in the buffer; this number's maximum possible value is set using M3dmapControl() with M_MAX_FRAMES. Note that the number of scanned laser lines actually accumulated can be lower than this maximum. Use M3dmapGetResult() with M_UNCORRECTED_DEPTH_MAP_SIZE_X, M_UNCORRECTED_DEPTH_MAP_SIZE_Y, and M_INTENSITY_MAP_BUFFER_TYPE to obtain the necessary buffer dimensions, data type, and depth.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCORRECTED_DEPTH_MAP		Copies the uncorrected depth map generated using acquired laser line data. In uncorrected depth maps, the gray value of a pixel is not scaled to the real depth in the world, and any shape distortion (due to the camera's angle) is not corrected.	c		e	f	g	a		i		j	k	l	m	n	b
	Image buffer identifier	Specifies the destination image buffer identifier. The destination image buffer must be a 1-band, 8-bit or 16-bit unsigned buffer. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. The X-size of the destination image buffer should be at least either the X-size or Y-size of the laser line images used to generate the data, depending on whether MimControl() with M_SCAN_LANE_DIRECTION is set to M_VERTICAL or M_HORIZONTAL, respectively. The Y-size of the destination image buffer must be at least as large as the number of scanned laser lines to accumulate in the buffer; this number's maximum possible value is set using M3dmapControl() with M_MAX_FRAMES. Note that the number of scanned laser lines actually accumulated can be lower than this maximum. Use M3dmapGetResult() with M_UNCORRECTED_DEPTH_MAP_SIZE_X, M_UNCORRECTED_DEPTH_MAP_SIZE_Y, and M_UNCORRECTED_DEPTH_MAP_BUFFER_TYPE to obtain the necessary buffer dimensions, data type, and depth. Use M_UNCORRECTED_DEPTH_MAP_FIXED_POINT to obtain the fixed point used to encode the laser line Y position.	c		e	f	g	a		i		j	k	l	m	n	b

3D reconstruction point cloud result buffer identifier	Specifies the identifier of a 3D reconstruction result buffer that stores results generated in M_CALIBRATED_CAMERA_LINEAR_MOTION mode.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTENSITY_MAP	Copies the intensity map generated using acquired laser line data. In an intensity map, the gray value of each pixel represents the luminous intensity of the laser line at this point. The size of the intensity map corresponds to the size of the uncorrected depth map obtained with M_UNCORRECTED_DEPTH_MAP.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the destination image buffer identifier. The destination image buffer must be a 1-band, 8-bit or 16-bit unsigned buffer. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. The X-size of the destination image buffer should be at least either the X-size or Y-size of the laser line images used to generate the data, depending on whether MimControl() with M_SCAN_LANE_DIRECTION is set to M_VERTICAL or M_HORIZONTAL, respectively. The Y-size of the destination image buffer must be at least as large as the number of scanned laser lines to accumulate in the buffer; this number's maximum possible value is set using M3dmapControl() with M_MAX_FRAMES. Note that the number of scanned laser lines actually accumulated can be lower than this maximum. Use M3dmapGetResult() with M_UNCORRECTED_DEPTH_MAP_SIZE_X, M_UNCORRECTED_DEPTH_MAP_SIZE_Y, and M_INTENSITY_MAP_BUFFER_TYPE to obtain the necessary buffer dimensions, data type, and depth.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_CLOUD +	Copies all points, preserving the point cloud's organizational type. All copied point coordinate values are respective to the relative coordinate system, unless the M_ABSOLUTE_COORDINATE_SYSTEM combination value is specified.	c		e	f	g	a		i		j	k	l	m	n	b
Container identifier	Specifies the destination container identifier. The destination container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container. If a range component and a confidence component already exist in the container, they will be overwritten. If the container is empty, range and confidence components are created to hold the copied information.	c		e	f	g	a		i		j	k	l	m	n	b

M_POINT_CLOUD_UNORGANIZED +	Copies only valid points, excluding points in the specified M_POINT_CLOUD_RESULT result buffer that are set to **M_INVALID_POINT**. The point cloud's organizational type is not preserved. Note that, when you use M_ALL to specify all point clouds in the M_POINT_CLOUD_RESULT result buffer, the point clouds are merged into a single point cloud, and then placed into the destination container. All copied point coordinate values are respective to the relative coordinate system, unless the M_ABSOLUTE_COORDINATE_SYSTEM combination value is specified.	c		e	f	g	a		i		j	k	l	m	n	b
Container identifier	Specifies the destination container identifier. The destination container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container. If a range component and a confidence component already exist in the container, they will be overwritten. If the container is empty, range and confidence components are created to hold the copied information.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCORRECTED_DEPTH_MAP	Copies the uncorrected depth map generated using acquired laser line data. In uncorrected depth maps, the gray value of a pixel is not scaled to the real depth in the world, and any shape distortion (due to the camera's angle) is not corrected.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the destination image buffer identifier. The destination image buffer must be a 1-band, 8-bit or 16-bit unsigned buffer. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. The X-size of the destination image buffer should be at least either the X-size or Y-size of the laser line images used to generate the data, depending on whether MimControl() with M_SCAN_LANE_DIRECTION is set to M_VERTICAL or M_HORIZONTAL, respectively. The Y-size of the destination image buffer must be at least as large as the number of scanned laser lines to accumulate in the buffer; this number's maximum possible value is set using M3dmapControl() with M_MAX_FRAMES. Note that the number of scanned laser lines actually accumulated can be lower than this maximum. Use M3dmapGetResult() with M_UNCORRECTED_DEPTH_MAP_SIZE_X, M_UNCORRECTED_DEPTH_MAP_SIZE_Y, and M_UNCORRECTED_DEPTH_MAP_BUFFER_TYPE to obtain the necessary buffer dimensions, data type, and depth. Use M_UNCORRECTED_DEPTH_MAP_FIXED_POINT to obtain the fixed point used to encode the laser line Y position.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when SrcResult3dmapId=3D reconstruction point cloud result buffer identifier:

M_POINT_CLOUD; M_POINT_CLOUD_UNORGANIZED].

You can add the following value to the above-mentioned values to specify to copy points with respect to the absolute coordinate system.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For copying points with respect to the absolute coordinate system																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ABSOLUTE_COORDINATE_SYSTEM	Specifies to copy points with respect to the absolute coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapDraw

Synopsis

Generate an image from a 3D reconstruction context or result buffer.

Syntax

```
void M3dmapDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID ContextOrResult3dmapId, //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_INT LabelOrIndex,          //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function draws specific features of a 3D reconstruction context, or result buffer.

> Unlike most other functions that modify an Aurora Imaging Library object, you can call this function concurrently from multiple threads on the same M_POINT_CLOUD_RESULT result buffer (ContextOrResult3dmapId) without using an M_MUTEX object. This is valid as long as the LabelOrIndex parameter of the concurrent calls is set to a different index.

Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. > Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextOrResult3dmapId

Specifies the identifier of a 3D reconstruction context or result buffer.

If you specify a 3D reconstruction context, it must have been previously allocated on the required system using M3dmapAlloc() and must have been successfully calibrated using M3dmapCalibrate().

If you specify a 3D reconstruction result buffer, it must have been previously allocated on the required system using M3dmapAllocResult(), and M3dmapAddScan() must have been called at least once on the result buffer.

[Zebra Radient eV-CL; Zebra Rapixo CL; Zebra Rapixo CoF; Zebra Rapixo CXP]

The 3D reconstruction context or result buffer must be allocated on the same system as the 2D graphics context (ContextGraId). If it is not, an error will occur.

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw.

If you specify to draw in an image buffer and you are drawing something from a 3D reconstruction context or result buffer, you can establish the minimum size with which to allocate the destination buffer, using M3dmapInquire() or M3dmapGetResult() with M_CAMERA_IMAGE_SIZE_X and M_CAMERA_IMAGE_SIZE_Y, respectively. In addition, you must not draw in an image buffer that has a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

If you specify a 2D graphics list, the 2D graphics list must have been previously allocated using MgraAllocList(). A 2D graphics list can only be used as a destination for certain operations.

Operation

Specifies what to draw in the destination image buffer or 2D graphics list.

The following operations can only draw in a 1-band destination image buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_REGION_INTERPOLATED	Draws all regions (pixels) of the laser line image that were interpolated due to missing data (gaps) in the calibration laser lines. Although the laser line can appear in these regions and be associated with a valid height, the heights will be less accurate since they are interpolated. This drawing operation is only available if ContextOrResult3dmapId is a 3D reconstruction context allocated with M_DEPTH_CORRECTION and calibrated using M3dmapCalibrate().	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_REGION_INVERTED	Draws all regions (pixels) of the laser line image where an inversion occurred. An inversion occurs when a reference plane associated with a lower height is found above a reference plane associated with a higher height. The laser line can still appear in these regions, however, results taken from these regions will be incorrect. This drawing operation is only available if ContextOrResult3dmapId is a 3D reconstruction context allocated with M_DEPTH_CORRECTION and calibrated using M3dmapCalibrate().	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_REGION_MISSING_DATA	Draws all regions (pixels) of the laser line image where the laser line cannot appear, because of missing data (gaps) in the calibration laser lines that could not be accounted for by interpolation. This drawing operation is only available if ContextOrResult3dmapId is a 3D reconstruction context allocated with M_DEPTH_CORRECTION and calibrated using M3dmapCalibrate().	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_REGION_UNCALIBRATED	Draws all regions (pixels) of the laser line image where the laser line cannot appear, because they are outside the calibrated region. This drawing operation is only available if ContextOrResult3dmapId is a 3D reconstruction context allocated with M_DEPTH_CORRECTION and calibrated using M3dmapCalibrate().	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_REGION_VALID	Draws all regions (pixels) of the laser line image where the laser line can appear and be associated with a valid height. This drawing operation is only available if ContextOrResult3dmapId is a 3D reconstruction context allocated with M_DEPTH_CORRECTION and calibrated using M3dmapCalibrate().	c		e	f	g	a		i		j	k	l	m	n	b

The following operations can draw in a 1-band or 3-band destination image buffer, or a 2D graphics list.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying an operation that can draw in a 2D graphics list																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_CALIBRATION_LINES	Draws all the fitted laser lines used for calibrating the 3D reconstruction setup. This drawing operation is only available if ContextOrResult3dmapId is a 3D reconstruction context allocated with M_CALIBRATED_CAMERA_LINEAR_MOTION and calibrated using M3dmapCalibrate().	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_CALIBRATION_PEAKS	Draws all the extracted laser lines used for calibrating the 3D reconstruction setup. This drawing operation is only available if ContextOrResult3dmapId is a 3D reconstruction context allocated with M_CALIBRATED_CAMERA_LINEAR_MOTION and calibrated using M3dmapCalibrate(). > If drawing in an image buffer, the 2D graphics context cannot have any zoom factors associated with it (MgraControl() with M_DRAW_ZOOM_X or M_DRAW_ZOOM_Y must be set to 1.0).	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_PEAKS_LAST	Draws the laser line that would have produced the most recent results stored in the 3D reconstruction result buffer using M3dmapAddScan(). This drawing operation is only available if ContextOrResult3dmapId is a 3D reconstruction result buffer, allocated using M3dmapAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b

LabelOrIndex

Reserved for future expansion and must be set to **M_DEFAULT**.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapDraw3d

Synopsis

Draw 3D annotations based on a laser line profiling 3D reconstruction context.

Syntax

```
AIL_INT64 M3dmapDraw3d(
    AIL_ID OperationDraw3dContext3dmapId, //in
    AIL_ID SrcReconContext3dmapId,        //in
    AIL_INT64 SrcIndex,                   //in
    AIL_ID DstList3dgraId,                //out
    AIL_INT64 DstParentLabel,             //in
    AIL_ID LaserPlaneTextureImageBufId,   //in
    AIL_INT64 ControlFlag                 //in
)
```

Description

This function draws 3D annotations (for example, the camera's coordinate system) based on a laser line profiling 3D reconstruction context, in a 3D graphics list. Set the draw operations and options for the draw using M3dmapControl(). Example operations include drawing the laser plane and the frustum of the camera's view.

Parameters

OperationDraw3dContext3dmapId

Specifies the identifier of the 3D draw context that specifies the annotations to draw and how to draw them. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 3D draw context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 3D draw context of the current Aurora Imaging Library application is used.	c		e	f	g	a		i		j	k	l	m	n	b
3D draw context identifier	Specifies a valid 3D draw context identifier, which you have allocated using M3dmapAlloc() with M_DRAW_3D_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b

SrcReconContext3dmapId

Specifies the identifier of the laser line profiling 3D reconstruction context, previously allocated using M3dmapAlloc() with M_LASER and M_CALIBRATED_CAMERA_LINEAR_MOTION. The 3D reconstruction setup must have been successfully calibrated using M3dmapCalibrate().

SrcIndex

Reserved for future expansion and must be set to **M_DEFAULT**.

DstList3dgraId

Specifies the identifier of the 3D graphics list in which to draw. You can specify a 3D graphics list that you have previously allocated using M3dgraAlloc(), or you can specify the identifier of the 3D display's internal graphics list (inquired using M3ddispInquire() with M_3D_GRAPHIC_LIST_ID).

DstParentLabel

Specifies the label of the 3D graphic in the 3D graphics list to be used as the annotation's parent.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the parent label																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ROOT_NODE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROOT_NODE	Specifies the top-most node of the 3D graphics list.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the label of the 3D graphic in the 3D graphics list. Label 0 is the 3D graphics list's root node.	c		e	f	g	a		i		j	k	l	m	n	b

LaserPlaneTextureImageBufId

Specifies the identifier of the image buffer containing the texture image, which you can apply to the drawn laser plane to enhance its visibility. This is typically the image buffer containing the grabbed image of the laser line.

Set this parameter to **M_NULL** if not used.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Return value

Returns the parent label of the graphics added to the graphics list.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapFree

Synopsis

Free a 3D reconstruction context, a 3D alignment context, a 3D reconstruction result buffer, or a 3D alignment result buffer.

Syntax

```
void M3dmapFree(
    AIL_ID M3dmapId //in
)
```

Description

This function deletes the specified 3D reconstruction context, 3D alignment context, 3D reconstruction result buffer, or 3D alignment result buffer and releases any memory associated with it.

All 3D reconstruction contexts, 3D alignment contexts, 3D reconstruction result buffers, and 3D alignment result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

M3dmapId

Specifies the identifier of the 3D reconstruction context, 3D alignement context, 3D reconstruction result buffer, or 3D alignment result buffer to free. These must have been successfully allocated (with M3dmapAlloc() or M3dmapAllocResult()) prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapGetResult

Synopsis

Get the specified type of result(s) from a 3D alignment or 3D reconstruction result buffer.

Syntax

```
void M3dmapGetResult(
    AIL_ID Result3dmapId, //in
    AIL_INT LabelOrIndex, //in
    AIL_INT64 ResultType, //in
    void *ResultArrayPtr  //out
)
```

Description

This function retrieves the result(s) of the specified type from a result buffer. For M_LASER_CALIBRATION_DATA, M_DEPTH_CORRECTED_DATA, and M_POINT_CLOUD_RESULT 3D reconstruction result buffers, results are available after using M3dmapAddScan() to fill the result buffer with the extracted laser line data. For 3D alignment result buffers, results are available after using M3dmapAlignScan().

An M_POINT_CLOUD_RESULT 3D reconstruction result buffer contains an array of distinct point clouds. You can retrieve results from an individual point cloud in the result buffer, from an aggregate of all point clouds in the result buffer, or from the result buffer itself using the LabelOrIndex parameter.

Parameters

Result3dmapId

Specifies the identifier of the 3D alignment or 3D reconstruction result buffer from which to retrieve results.

LabelOrIndex

Specifies the point cloud(s) in the specified 3D reconstruction result buffer, or the entire result buffer itself, from which to get results. Only 3D reconstruction result buffers allocated using M_POINT_CLOUD_RESULT have point clouds that can be specified using this parameter. For all other types of 3D reconstruction result buffers, set this parameter to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the point cloud(s) or point cloud container																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_POINT_CLOUD_INDEX(AIL_INT PointCloudIndex)	Specifies the array index of a point cloud in the specified 3D reconstruction result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PointCloudIndex Specifies the array index of an existing point cloud.															
M_POINT_CLOUD_LABEL(AIL_INT PointCloudLabel)	Specifies the label for a point cloud in the specified 3D reconstruction result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															

	PointCloudLabel Specifies the label of an existing point cloud.														
M_ALL	Specifies to get results about all point clouds in the specified 3D reconstruction result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to get results about the point cloud container (3D reconstruction result buffer allocated using M_POINT_CLOUD_RESULT).	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to get.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the results.

When LabelOrIndex is set to M_ALL, you must specify the address of an array with as many elements as point clouds in the point cloud container. You can inquire the number of point clouds using M3dmapInquire() set to M_NUMBER_OF_POINT_CLOUDS.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **ResultArrayPtr** parameter.

- For retrieving results from a 3D reconstruction result buffer of typeM_POINT_CLOUD_RESULT
- For retrieving results from a 3D reconstruction result buffer of typeM_DEPTH_CORRECTED_DATA
- For retrieving results from a 3D reconstruction result buffer of type M_LASER_CALIBRATION_DATA or M_DEPTH_CORRECTED_DATA, or individual point clouds.
- For retrieving results from a 3D alignment result buffer.

To retrieve a result from an M_POINT_CLOUD_RESULT 3D reconstruction result buffer, the ResultType parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results from a 3D reconstruction result buffer of typeM_POINT_CLOUD_RESULT

ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_HAS_FEATURE +	Retrieves whether the specified point cloud(s) have the specified feature (such as position or intensity data). The feature is specified using the combination value. When the LabelOrIndex parameter is set to M_POINT_CLOUD_INDEX() or M_POINT_CLOUD_LABEL(), M_HAS_FEATURE returns whether the specified point cloud has the feature. When the LabelOrIndex parameter is set to M_GENERAL, M_HAS_FEATURE returns whether all the point clouds in the specified result buffer have the feature. When the LabelOrIndex parameter is set to M_ALL, M_HAS_FEATURE returns an array where the value of each element in the array is whether the point cloud at the corresponding index has the feature. You must specify a combination value from the following table: • To determine whether the feature exists in the specified location																
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE ┃ array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the specified point cloud(s) do not have the feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the specified point cloud(s) do have the feature.	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_OF_3D_POINTS +	Retrieves the number of 3D points extracted from laser line data and stored in the specified point cloud(s). When the LabelOrIndex parameter is set to M_POINT_CLOUD_INDEX() or M_POINT_CLOUD_LABEL(), M_NUMBER_OF_3D_POINTS returns the number of points in the specified point cloud. When the LabelOrIndex parameter is set to M_GENERAL, M_NUMBER_OF_3D_POINTS returns the total number of points in all the point clouds in the specified result buffer. When the LabelOrIndex parameter is set to M_ALL, M_NUMBER_OF_3D_POINTS returns an array where the value of each element in the array is the number of points in the point cloud at the corresponding index.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* | *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of 3D points.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOTAL_DISPLACEMENT_Y +	Retrieves the total displacement along the Y-axis (typically the conveyor) from the first call to M3dmapAddScan(). If the value of M_SCAN_SPEED (M3dmapControl()) has remained constant since the last call to M3dmapClear(), M_TOTAL_DISPLACEMENT_Y is equivalent to M_SCAN_SPEED x (number of calls to M3dmapAddScan()-1). M_TOTAL_DISPLACEMENT_Y is only available when the LabelOrIndex parameter is set to M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_HAS_FEATURE.

You must add one of the following values to the above-mentioned value to determine whether the feature exists in the specified location.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the feature to check																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTENSITY	Specifies to determine if intensity information exists in the specified point cloud(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION	Specifies to determine if position information exists in the specified point cloud(s).	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_NUMBER_OF_3D_POINTS.

You can add the following value to the above-mentioned value to specify which 3D points should be included when

returning the number of points.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For modifying the number of points returned																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EXCLUDE_INVALID_POINTS +	Specifies to exclude all points in the specified point cloud(s) that are set to **M_INVALID_POINT**. Only valid points are included in the calculation.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_NUMBER_OF_3D_POINTS; M_EXCLUDE_INVALID_POINTS.

You can add the following value to the above-mentioned values to specify to return the points of the last scan only.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For including the last scan only																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LAST_SCAN	Specifies to return only those points generated during the last call to M3dmapAddScan().	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a result about partially corrected depth maps from an M_DEPTH_CORRECTED_DATA 3D reconstruction result buffer, the ResultType parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results from a 3D reconstruction result buffer of typeM_DEPTH_CORRECTED_DATA																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_PARTIALLY_CORRECTED_DEPTH_MAP_BUFFER_TYPE +		Retrieves the data type and depth that an image buffer should have to store the partially corrected depth map.															
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 8	Specifies that the image buffer should be an 8-bit unsigned buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 16	Specifies that the image buffer should be a 16-bit unsigned buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_PARTIALLY_CORRECTED_DEPTH_MAP_SIZE_X +	Retrieves the X-size, in pixels, that an image buffer should have to store the partially corrected depth map. If the MimControl() M_SCAN_LANE_DIRECTION control type setting is set to M_VERTICAL, this is equal to the X-size of the image buffer passed to M3dmapAddScan(). If the MimControl() M_SCAN_LANE_DIRECTION control type setting is set to M_HORIZONTAL, this is equal to the Y-size of the image buffer passed to M3dmapAddScan().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PARTIALLY_CORRECTED_DEPTH_MAP_SIZE_Y +	Retrieves the Y-size, in pixels, that an image buffer should have to store the partially corrected depth map. This is equal to the number of times M3dmapAddScan() was called or the value of the M_MAX_FRAMES control, whichever is smaller.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a result from a 3D reconstruction result buffer of type M_LASER_CALIBRATION_DATA or M_DEPTH_CORRECTED_DATA, or from an individual point cloud, the ResultType parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results from a 3D reconstruction result buffer of type M_LASER_CALIBRATION_DATA or M_DEPTH_CORRECTED_DATA, or individual point clouds.

ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_CAMERA_IMAGE_SIZE_X +	Retrieves the X-size of the image buffer from which the last laser line was extracted using M3dmapAddScan().															

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CAMERA_IMAGE_SIZE_Y +		Retrieves the Y-size of the image buffer from which the last laser line was extracted using M3dmapAddScan().	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INTENSITY_MAP_BUFFER_TYPE +		Retrieves the data type and depth that an image buffer should have to store the intensity map. The returned depth and data type are the same as those of the image buffer, passed to M3dmapAddScan(), containing the grabbed image of the laser line.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 8	Specifies that the image buffer should be an 8-bit unsigned buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 16	Specifies that the image buffer should be a 16-bit unsigned buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_MISSING_DATA_LAST_SCAN +		Retrieves the number of points (pixels) with unknown values in the last laser line extracted using M3dmapAddScan().	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_LANE_DIRECTION +		Retrieves whether laser line detection was performed vertically or horizontally.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_UNCORRECTED_DEPTH_MAP_BUFFER_TYPE +		Retrieves the data type and depth that an image buffer should have to store the uncorrected depth map.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 8	Specifies that the image buffer should be an 8-bit unsigned buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 16	Specifies that the image buffer should be a 16-bit unsigned buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCORRECTED_DEPTH_MAP_FIXED_POINT +		Retrieves the number of fractional bits used for the uncorrected depth map.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCORRECTED_DEPTH_MAP_SIZE_X +		Retrieves the X-size, in pixels, that an image buffer should have to store the uncorrected depth map. Note that M_UNCORRECTED_DEPTH_MAP_SIZE_X is also the X-size for storing the intensity map.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCORRECTED_DEPTH_MAP_SIZE_Y +		Retrieves the Y-size, in pixels, that an image buffer should have to store the uncorrected depth map. Note that M_UNCORRECTED_DEPTH_MAP_SIZE_Y is also the Y-size for storing the intensity map.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a result from a 3D alignment result buffer, the ResultType parameter can be set to one of the following values:

> Note that aside from M_STATUS, these result types are only available after a successfully completed alignment operation (when M_STATUS is M_COMPLETE) or if a status warning is returned (when M_STATUS is

M_RESULTS_Z_NEED_BACKGROUND, M_MAYBE_IMPRECISE, or M_CHAMFER_NOT_FOUND).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results from a 3D alignment result buffer.

ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_3D_SCALE_Y	Retrieves the Y-scaling factor necessary to correct the Y-coordinates of the point clouds. Scale distortions in the Y-direction cause shrinking or stretching of the object scanned by the 3D profile sensor.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SHEAR_X	Retrieves by how much to offset the X-coordinates of the point cloud from the X-coordinates of its previous row to correct the shear in X. Using MbufControlContainer(), you can set M_3D_SHEAR_X of the range component to this value, prior to calling MbufConvert3d(), to correct the shear in X. Shear distortions in the X-direction cause skewing of the object scanned by the 3D profile sensor.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SHEAR_Z	Retrieves by how much to offset the Z-coordinates of the point cloud from the Z-coordinates of its previous row to correct the shear in Z. Using MbufControlContainer(), you can set M_3D_SHEAR_Z of the range component to this value, prior to calling MbufConvert3d(), to correct the shear in Z. Shear distortions in the Z-direction cause inaccurate object thickness scanned by the 3D profile sensor.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_HOLES_FOUND	Retrieves the detection status of the two holes on the alignment disk. This result verifies that the alignment disk's holes have been taken into consideration during the alignment operation. This result is only available when M_OBJECT_SHAPE is M_DISK.	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the two holes are not detected on the alignment disk. The two holes have not been taken into consideration during the alignment operation. > Note that if the two holes are not detected on the alignment disk, M_Y_MIRRORED will not be correct.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the two holes are detected on the alignment disk. The two holes have been taken into consideration during the alignment operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_PYRAMID_CORNERS_X		Retrieves the uncorrected X-coordinate for each of the corners of the truncated pyramid on the alignment pyramid. The order in which the uncorrected X-coordinates are returned is not guaranteed; however, the results are returned uniformly across the M_PYRAMID_CORNERS_X, M_PYRAMID_CORNERS_Y, and M_PYRAMID_CORNERS_Z result types. This result is only available when M_OBJECT_SHAPE is M_PYRAMID.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PYRAMID_CORNERS_Y		Retrieves the uncorrected Y-coordinate for each of the corners of the truncated pyramid on the alignment pyramid. The order in which the uncorrected Y-coordinates are returned is not guaranteed, however the results are returned uniformly across the M_PYRAMID_CORNERS_X, M_PYRAMID_CORNERS_Y, and M_PYRAMID_CORNERS_Z result types. This result is only available when M_OBJECT_SHAPE is M_PYRAMID.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_PYRAMID_CORNERS_Z	Retrieves the uncorrected Z-coordinate for each of the corners of the truncated pyramid on the alignment pyramid. The order in which the uncorrected Z-coordinates are returned is not guaranteed, however the results are returned uniformly across the M_PYRAMID_CORNERS_X, M_PYRAMID_CORNERS_Y, and M_PYRAMID_CORNERS_Z result types. This result is only available when M_OBJECT_SHAPE is M_PYRAMID.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_RMS_ERROR	Retrieves the root-mean-square (RMS) error. The RMS error is calculated using the distance measured between the alignment pyramid's aligned corners and their target corners. This result is only available when M_OBJECT_SHAPE is M_PYRAMID.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SENSOR_PITCH	Retrieves the angle of the pitch (the angle of rotation around the X-axis) of the 3D profile sensor, in degrees; this is the pitch angle if the required rotation around the Z-axis is applied (M_SENSOR_YAW_BEFORE_PITCH) before applying a rotation around the X-axis. This is not the angle necessary to remove the shear. To retrieve the angle of the pitch of the 3D profile sensor, necessary to remove shear in the Z-direction, use M_3D_SHEAR_Z.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SENSOR_PITCH_BEFORE_YAW	Retrieves the angle of the pitch (the angle of rotation around the X-axis) of the 3D profile sensor, in degrees; this is the pitch angle if the rotation around the X-axis is applied before applying the required rotation around the Z-axis (M_SENSOR_YAW). This is not the angle necessary to remove the shear. To retrieve the angle of the pitch of the 3D profile sensor, necessary to remove shear in the Z-direction, use M_3D_SHEAR_Z.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_SENSOR_YAW	Retrieves the angle of the yaw (the angle of rotation around the Z-axis) of the 3D profile sensor, in degrees; this is the yaw angle if the required rotation around the X-axis is applied (M_SENSOR_PITCH_BEFORE_YAW) before applying a rotation around the Z-axis. This is not the angle necessary to remove the shear. To retrieve the angle of the yaw of the 3D profile sensor, necessary to remove shear in the X-direction, use M_3D_SHEAR_X.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SENSOR_YAW_BEFORE_PITCH	Retrieves the angle of the yaw (the angle of rotation around the Z-axis) of the 3D profile sensor, in degrees; this is the yaw angle if the rotation around the Z-axis is applied before applying the required rotation around the X-axis (M_SENSOR_PITCH). This is not the angle necessary to remove the shear. To retrieve the angle of the yaw of the 3D profile sensor, necessary to remove shear in the X-direction, use M_3D_SHEAR_X.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS	Retrieves the status of the alignment operation. This result is always available for retrieval.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_POINTS_COLLINEAR	Specifies that the alignment operation failed. This occurs when all the points found create a line.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAMFER_NOT_FOUND	Specifies that the alignment pyramid's chamfer was not found. When this occurs, the step direction (the sign of M_3D_SCALE_Y) could be wrong and/or the rotation around the Z-axis might be off by a factor of 90 degrees. > This result is only available when M_OBJECT_SHAPE is M_PYRAMID.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPLETE	Specifies that the alignment operation completed successfully.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISK_EDGES_NOT_FOUND	Specifies that the alignment disk's edges could not be found. This occurs when the disk is clipped during the scan, or the edges of the disk were not clear enough to determine the start and end of the disk. > This result is only available when M_OBJECT_SHAPE is M_DISK.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISK_NOT_FOUND	Specifies that the alignment disk could not be found. This occurs when the scan is too noisy for the disk to be found. > This result is only available when M_OBJECT_SHAPE is M_DISK.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID_STEP_LENGTH	Specifies that the step length is invalid. This occurs when the user-defined step length is set to an invalid value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MATHEMATICAL_EXCEPTION	Specifies that the alignment pyramid could not converge to a feasible alignment solution. This occurs when the optimizer throws an exception. This can be resolved by properly following the design constraints of the alignment pyramid, or by setting M_STEP_LENGTH_MODE to a value that is not M_UNKNOWN. > This result is only available when M_OBJECT_SHAPE is M_PYRAMID.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAYBE_IMPRECISE	Specifies that the alignment pyramid is not precise enough (scan resolution). This typically occurs when the chamfer is not detected. This can be resolved by properly following the design constraints of the alignment pyramid, and ensuring that the chamfer is visible in the scan. > This result is only available when M_OBJECT_SHAPE is M_PYRAMID.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NOT_ENOUGH_VALID_DATA	Specifies that the alignment operation failed. This occurs when there are not enough valid points.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NOT_INITIALIZED	Specifies that the alignment result buffer was not used in a call to M3dmapAlignScan(), and contains no results.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PYRAMID_NOT_FOUND	Specifies that the alignment pyramid could not be found. This occurs when its truncated pyramid's faces or corners are not found. > This result is only available when M_OBJECT_SHAPE is M_PYRAMID.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RESULTS_Z_NEED_BACKGROUND	Specifies that the required background was not found. This occurs when the background is outside the FOV of the 3D profile sensor. In this case, the background will be estimated with respect to M_CAMERA_TILT_X set to M_ZERO. > Note that in this case, M_3D_SHEAR_Z and M_SENSOR_PITCH will not be accurate. The presence of a background is also required for M_ALIGN_Z_DIRECTION, M_ALIGN_Z_POSITION, and M_CAMERA_TILT_X in M3dmapControl().	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_LENGTH		Retrieves the norm of the calculated step vector. This vector will include shear in the X and Z-axes, and scale in the Y-axis if they are present. When the 3D profile sensor is parallel to the plane of motion (conveyor) and has no shear in the X or Z-axes, the step length will be the distance between scan lines in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_MIRRORED		Retrieves whether the alignment will cause a mirroring of the Y-axis. If M_HOLES_FOUND returns M_FALSE and the two holes are not detected on the alignment disk, the result of M_Y_MIRRORED will be incorrect. This result is only available when M_OBJECT_SHAPE is M_DISK.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the Y-axis is not mirrored.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the Y-axis is mirrored. > After alignment, the scan will be reflected around the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For retrieving results from a 3D reconstruction result buffer of type M_LASER_CALIBRATION_DATA or M_DEPTH_CORRECTED_DATA, or individual point clouds.; and for the following values: M_HAS_FEATURE; M_NUMBER_OF_3D_POINTS.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																	
ResultType combination value		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_NB_ELEMENTS		Retrieves the required array size (number of elements) to store the returned values.															
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For retrieving results from a 3D reconstruction result buffer of typeM_POINT_CLOUD_RESULT
- For retrieving results from a 3D reconstruction result buffer of typeM_DEPTH_CORRECTED_DATA
- For retrieving results from a 3D reconstruction result buffer of type M_LASER_CALIBRATION_DATA or M_DEPTH_CORRECTED_DATA, or individual point clouds.

the values listed in all the parameter association tables except **For retrieving results from a 3D alignment result buffer.**.

You can add the following value to the above-mentioned values to determine whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_AVAILABLE		Retrieves whether the requested result type is available for retrieval.															
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For retrieving results from a 3D reconstruction result buffer of typeM_POINT_CLOUD_RESULT
- For retrieving results from a 3D reconstruction result buffer of typeM_DEPTH_CORRECTED_DATA
- For retrieving results from a 3D reconstruction result buffer of type M_LASER_CALIBRATION_DATA or M_DEPTH_CORRECTED_DATA, or individual point clouds.

the values listed in all the parameter association tables except **For retrieving results from a 3D alignment result buffer.**.

You can add one of the following values to the above-mentioned values to cast the requested results to the required

data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																	
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.																
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]* **|** *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]* **|** *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]* **|** *address of an AIL_INT16*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapInquire

Synopsis

Inquire about a 3D alignment context, profiling 3D reconstruction context, draw 3D reconstruction context, or 3D reconstruction result buffer.

Syntax

```
AIL_INT M3dmapInquire(
    AIL_ID ContextOrResult3dmapId, //in
    AIL_INT LabelOrIndex,          //in
    AIL_INT64 InquireType,         //in
    void *UserVarPtr               //out
)
```

Description

This function inquires about a specified setting of a 3D alignment context, profiling 3D reconstruction context, draw 3D reconstruction context, or 3D reconstruction result buffer.

If the inquired setting is set to M_DEFAULT, **M3dmapInquire**() will return M_DEFAULT. To inquire the actual default value, add M_DEFAULT to the InquireType parameter.

An M_POINT_CLOUD_RESULT 3D reconstruction result buffer contains an array of distinct point clouds. You can retrieve results from an individual point cloud in the result buffer, from an aggregate of all point clouds in the result buffer, or from the result buffer itself using the LabelOrIndex parameter.

Parameters

ContextOrResult3dmapId

Specifies the identifier of the 3D alignment context, profiling 3D reconstruction context, draw 3D reconstruction context, or 3D reconstruction result buffer about which to inquire information.

LabelOrIndex

Specifies the point cloud(s) in the result buffer about which to inquire, or specifies the entire result buffer itself. Only 3D reconstruction result buffers allocated using M_POINT_CLOUD_RESULT have point clouds that can be specified using this parameter. For other types of 3D reconstruction contexts and result buffers, as well as for 3D draw contexts, set this parameter to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the point cloud(s) or point cloud result buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_POINT_CLOUD_INDEX(AIL_INT PointCloudIndex)	Specifies the index of a point cloud in the specified 3D reconstruction result buffer about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PointCloudIndex Specifies the index of an existing point cloud.															

M_POINT_CLOUD_LABEL(AIL_INT PointCloudLabel)	Specifies the label of a point cloud in the specified 3D reconstruction result buffer about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PointCloudLabel Specifies the label of an existing point cloud.															
M_ALL	Specifies to inquire all point clouds in the specified 3D reconstruction result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to inquire the specified 3D reconstruction result buffer allocated using M_POINT_CLOUD_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the setting to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **M3dmapInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

When LabelOrIndex is set to M_ALL, you must specify the address of an array with as many elements as point clouds in the point cloud container. You can inquire the number of point clouds using **M3dmapInquire**() set to M_NUMBER_OF_POINT_CLOUDS.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about the system

- For a 3D alignment context
- For 3D reconstruction contexts used for laser line profiling
- For profiling 3D reconstruction contexts set to M_CALIBRATED_CAMERA_LINEAR_MOTION
- For profiling 3D reconstruction contexts set to M_DEPTH_CORRECTION
- For a 3D reconstruction result buffer set to M_POINT_CLOUD_RESULT
- For a point cloud
- For a draw 3D reconstruction context
- For inquiring the maximum number of frames

To inquire about the system on which the profiling 3D reconstruction context, draw 3D reconstruction context, or result buffer has been allocated, set the InquireType parameter to the value below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the system																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_OWNER_SYSTEM +		Inquires the identifier of the system on which the profiling 3D reconstruction context, draw 3D reconstruction context, or result buffer has been allocated. SET SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

For a 3D alignment context, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a 3D alignment context																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_ALIGN_X_POSITION		Inquires whether to calculate the transformation required to align the origin of the working coordinate system with the center of the alignment object along the X-axis. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_SAME.	c		e	f	g	a		i		j	k	l	m	n	b

	M_OBJECT_ORIGIN	Specifies to calculate the transformation to align the origin of the working coordinate system with the origin of the alignment object along the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME	Specifies not to calculate the transformation to align the origin of the working coordinate system along the X-axis. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ALIGN_XY_DIRECTION		Inquires the direction of the X- and Y-axes of the working coordinate system relative to its original direction. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to calculate the transformation to align with default direction; this is specific to the type of alignment object. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_OBJECT_REVERSE_X	Specifies to calculate the transformation to align the X-axis of the working coordinate system with the reverse direction of the X-axis of the alignment pyramid. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_OBJECT_REVERSE_Y	Specifies to calculate the transformation to align the Y-axis of the working coordinate system with the reverse direction of the Y-axis of the alignment pyramid. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_OBJECT_SAME_X	Specifies to calculate the transformation to align the X-axis of the working coordinate system with the direction of the X-axis of the alignment pyramid. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_OBJECT_SAME_Y	Specifies to calculate the transformation to align the Y-axis of the working coordinate system with the direction of the Y-axis of the alignment pyramid. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_REVERSE_X	Specifies to calculate the transformation required to reverse the direction of the X-axis of the working coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_REVERSE_Y	Specifies to calculate the transformation required to reverse the direction of the Y-axis of the working coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME_X	Specifies to maintain the current direction of the X-axis of the working coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_SAME_Y	Specifies to maintain the current direction of the Y-axis of the working coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ALIGN_Y_POSITION	Inquires whether to calculate the transformation required to align the origin of the working coordinate system with the center of the alignment object along the Y-axis. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SAME.	c		e	f	g	a		i		j	k	l	m	n	b
M_OBJECT_ORIGIN	Specifies to calculate the transformation to align the origin of the working coordinate system with the origin of the alignment object along the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME	Specifies not to calculate the transformation to align the origin of the working coordinate system along the Y-axis. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ALIGN_Z_DIRECTION	Inquires the direction of the Z-axis of the working coordinate system. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SAME.	c		e	f	g	a		i		j	k	l	m	n	b
M_REVERSE	Specifies to calculate the transformation required to reverse the direction of the Z-axis, relative to its original direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME	Specifies not to calculate the transformation required to change the direction of the Z-axis. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_Z_DOWN	Specifies to calculate the transformation required to point the Z-axis downwards. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_Z_UP	Specifies to calculate the transformation required to point the Z-axis upwards. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ALIGN_Z_POSITION	Inquires whether to calculate the transformation required to align the origin of the working coordinate system with respect to the top or bottom of the alignment object. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_OBJECT_BOTTOM.	c		e	f	g	a		i		j	k	l	m	n	b

	M_OBJECT_BOTTOM	Specifies to calculate the transformation to align the origin of the working coordinate system with the bottom of the alignment object along the Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	M_OBJECT_TOP	Specifies to calculate the transformation to align the origin of the working coordinate system with the top of the alignment object along the Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME	Specifies not to calculate the transformation to align the origin of the working coordinate system along the Z-axis. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CAMERA_TILT_X		Inquires whether to enable camera tilt correction around the X-axis, and in which direction to enable the correction. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ZERO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NEGATIVE	Specifies that the camera tilt is in the negative direction around the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	M_POSITIVE	Specifies that the camera tilt is in the positive direction around the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ZERO	Specifies that there is no camera tilt around the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIAMETER		Inquires the diameter of the alignment disk. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_UNKNOWN.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNKNOWN	Specifies that the diameter of the alignment disk is unknown.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the diameter of the alignment disk.	c		e	f	g	a		i		j	k	l	m	n	b
M_HEIGHT		Inquires the height of the alignment object that was specified with M_OBJECT_SHAPE. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

Value >= 0.0	Specifies the height of the alignment object.	c		e	f	g	a		i		j	k	l	m	n	b
M_OBJECT_SHAPE	Inquires the type of alignment object. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISK.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISK	Specifies a disk-type object for alignment (alignment disk). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FLAT_SURFACE	Specifies a flat-surface-type object for alignment (alignment surface). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PYRAMID	Specifies a pyramid-type object for alignment (alignment pyramid). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PYRAMID_BASE_LENGTH	Inquires the length of the pyramid's bottom base for the alignment pyramid. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_UNKNOWN.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN	Specifies that the length of the truncated pyramid's bottom base is unknown.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the length of the truncated pyramid's bottom base.	c		e	f	g	a		i		j	k	l	m	n	b
M_PYRAMID_TOP_FACE_LENGTH	Inquires the length of the pyramid's top base (top face) for the alignment pyramid. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_UNKNOWN.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN	Specifies that the length of the truncated pyramid's top base is unknown.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the length of the truncated pyramid's top base.	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_LENGTH	Inquires the step length (scan step). The step length is distance between the scan lines. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the step length value.	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_LENGTH_MODE		Inquires the step length (scan step) mode. The step length is distance between the scan lines. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_UNKNOWN.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL_SIZE_Y	Specifies that the step length is provided with the container that stores the point cloud of the alignment object. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNKNOWN	Specifies that the step length is unknown. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies that the step length is explicitly set with M_STEP_LENGTH. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For a profiling 3D reconstruction context of type M_LASER, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For 3D reconstruction contexts used for laser line profiling																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_CALIBRATION_DEPTHS +	Inquires the calibration depths used to calibrate the profiling 3D reconstruction context. These are the values given specified using M3dmapControl() with M_CORRECTED_DEPTH before each call to M3dmapAddScan().																
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the calibration depths used to calibrate the profiling 3D reconstruction context.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATION_STATUS +	Inquires the status of the profiling 3D reconstruction calibration.	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED	Specifies that a successful call to M3dmapCalibrate() has been made and the context can now be used to produce calibrated data.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_OPTIMIZATION_ERROR	Specifies that the global optimization phase of M3dmapCalibrateMultiple() failed because of a mathematical exception.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERNAL_ERROR	Specifies that an unexpected error occurred.	c		e	f	g	a		i		j	k	l	m	n	b
M_LASER_LINE_NOT_DETECTED	Specifies that the laser line could not be extracted from the image passed as input to M3dmapAddScan().	c		e	f	g	a		i		j	k	l	m	n	b
M_MATHEMATICAL_EXCEPTION	Specifies that M3dmapCalibrate() could not calibrate properly. Verify that all function parameters, control type settings, and images are consistent.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_ENOUGH_MEMORY	Specifies that there was not enough memory for M3dmapCalibrate() to complete its task.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_INITIALIZED	Specifies that the context is not calibrated. M3dmapCalibrate() has not been called.	c		e	f	g	a		i		j	k	l	m	n	b
M_CAMERA_IMAGE_SIZE_X +	Inquires the X-size of the image(s) with which this profiling 3D reconstruction context was calibrated. This inquire type is valid only after a successful call to M3dmapCalibrate() on this profiling 3D reconstruction context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the width of the image with which this profiling 3D reconstruction context was calibrated. The width is rounded to the nearest integer.	c		e	f	g	a		i		j	k	l	m	n	b
M_CAMERA_IMAGE_SIZE_Y +	Inquires the Y-size of the image(s) with which this profiling 3D reconstruction context was calibrated. This inquire type is valid only after a successful call to M3dmapCalibrate() on this profiling 3D reconstruction context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value > 0	Specifies the height of the image with which this profiling 3D reconstruction context was calibrated. The height is rounded to the nearest integer.	c		e	f	g	a		i		j	k	l	m	n	b
M_CORRECTED_DEPTH +		Inquires depth information about the next laser line image added. For a profiling 3D reconstruction context allocated using M3dmapAlloc() with M_DEPTH_CORRECTION, this inquire type inquires the gray level corresponding to the depth represented by the next laser line image added (using M3dmapAddScan()) when calibrating your 3D reconstruction setup. For a profiling 3D reconstruction context allocated using M3dmapAlloc() with M_CALIBRATED_CAMERA_LINEAR_MOTION, this inquire type inquires the depth in world units represented by the next laser line image added (using M3dmapAddScan()) when calibrating your 3D reconstruction setup. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *GrayValue* <= 254	Specifies the gray level that will be used to represent the height of the next reference plane, when M3dmapAlloc() is set to M_DEPTH_CORRECTION. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *GrayValue* <= 65534	Specifies the gray level that will be used to represent the height of the next reference plane, when M3dmapAlloc() is set to M_DEPTH_CORRECTION. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	ZCoordinateValue	Specifies the Z-coordinate (in world units) of the next reference plane. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACTION_FIXED_POINT +		Inquires the number of binary digits used for the fractional part of the gray level in the uncorrected depth map, when using M3dmapAddScan() with M_LINE_ALREADY_EXTRACTED. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b

0 <= *Value* <= 7	Specifies the number of binary digits used for the fractional part of gray level values.	c		e	f	g	a		i		j	k	l	m	n	b
M_LASER_CONTEXT_TYPE +	Inquires the profiling 3D reconstruction mode, which determines whether the context can associate with a camera calibration and create a fully corrected depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED_CAMERA_LINEAR_MOTION	Specifies that the 3D reconstruction context will include camera calibration information and depth correction information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_CORRECTION	Specifies that the 3D reconstruction context will include depth correction information, but will not include camera calibration information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCATE_PEAK_1D_CONTEXT_ID +	Inquires the identifier of the internal locate peak 1D context within the profiling 3D reconstruction context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CALIBRATION_DEPTHS +	Inquires the number of calibration depths used to calibrate the profiling 3D reconstruction context. This inquire type is valid only after a successful call to M3dmapCalibrate().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of calibration depths used to calibrate the profiling 3D reconstruction context. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b

For profiling 3D reconstruction contexts set to M_CALIBRATED_CAMERA_LINEAR_MOTION, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For profiling 3D reconstruction contexts set to M_CALIBRATED_CAMERA_LINEAR_MOTION																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	

M_ASSUMED_PERPENDICULAR_TO_MOTION +		Inquires whether the laser plane was assumed to be perpendicular to the object's motion because a single reference plane was provided.													
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the laser plane was not assumed perpendicular to the object's motion.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the laser plane was assumed perpendicular to the object's motion.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACTION_CHILD_OFFSET_X +		Inquires the X-offset that M3dmapAddScan() assumes the laser line image buffer to have relative to the top-left pixel of the image buffer used during camera calibration. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the X-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACTION_CHILD_OFFSET_Y +		Inquires the Y-offset that M3dmapAddScan() assumes the laser line image buffer to have relative to the top-left pixel of the image buffer used during camera calibration. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the Y-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACTION_RANGE_Z +		Inquires the mode that helps determine the range of valid Z-coordinates for extracted points when using M3dmapAddScan(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_GREATER	Specifies that the Z-axis range is defined by a lower limit corresponding to M_EXTRACTION_RANGE_Z_LIMIT1. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_IN_RANGE	Specifies that the Z-axis range is defined by the inside range of a lower limit and a upper limit corresponding to M_EXTRACTION_RANGE_Z_LIMIT1 and M_EXTRACTION_RANGE_Z_LIMIT2. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies that the range covers the entire Z-axis, so all points are kept.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LESS	Specifies that the Z-axis range is defined by a upper limit corresponding to M_EXTRACTION_RANGE_Z_LIMIT1. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_OUT_RANGE	Specifies that the Z-axis range is defined by the outside range of a lower limit and a upper limit corresponding to M_EXTRACTION_RANGE_Z_LIMIT1 and M_EXTRACTION_RANGE_Z_LIMIT2. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACTION_RANGE_Z_LIMIT1 +		Inquires the first limit value that determines the range of valid Z-coordinates for extracted points. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the first limit value, which can be either the lower limit or upper limit.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACTION_RANGE_Z_LIMIT2 +		Inquires the first limit value that determines the range of valid Z-coordinates for extracted points. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the second limit value, which can be either the lower limit or upper limit.	c		e	f	g	a		i		j	k	l	m	n	b

M_FIT_RMS_ERROR +		Inquires the root mean squared error. This inquire type returns the root mean squared error of all inlier 3D points used to fit the laser plane to the calibration laser lines upon calling M3dmapCalibrate(). For more information, consult the Inspecting laser line calibration subsection of the Calibrating your 3D reconstruction setup to create a point cloud section of Chapter 46: 3D reconstruction using laser line profiling. In this case, this inquire type is only supported after a successful call to M3dmapCalibrate().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the root mean square error.	c		e	f	g	a		i		j	k	l	m	n	b
M_LASER_PLANE_A +		Inquires the coefficient *a* of the laser plane equation, ax + by + cz + d = 0. This inquire type is only supported after a successful call to M3dmapCalibrate() or M3dmapCalibrateMultiple(). Note that the variables *x*, *y*, *z*, and *d* are expressed in world units in the absolute coordinate system, while the coefficients *a*, *b*, and *c* are dimensionless.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the coefficient *a* of the laser plane equation.	c		e	f	g	a		i		j	k	l	m	n	b
M_LASER_PLANE_B +		Inquires the coefficient *b* of the laser plane equation, ax + by + cz + d = 0. This inquire type is only supported after a successful call to M3dmapCalibrate() or M3dmapCalibrateMultiple(). Note that the variables *x*, *y*, *z*, and *d* are expressed in world units in the absolute coordinate system, while the coefficients *a*, *b*, and *c* are dimensionless.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the coefficient *b* of the laser plane equation.	c		e	f	g	a		i		j	k	l	m	n	b
M_LASER_PLANE_C +		Inquires the coefficient *c* of the laser plane equation, ax + by + cz + d = 0. This inquire type is only supported after a successful call to M3dmapCalibrate() or M3dmapCalibrateMultiple(). Note that the variables *x*, *y*, *z*, and *d* are expressed in world units in the absolute coordinate system, while the coefficients *a*, *b*, and *c* are dimensionless.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the coefficient *c* of the laser plane equation.	c		e	f	g	a		i		j	k	l	m	n	b
M_LASER_PLANE_D +		Inquires the coefficient *d* of the laser plane equation, ax + by + cz + d = 0. This inquire type is only supported after a successful call to M3dmapCalibrate() or M3dmapCalibrateMultiple(). Note that the variables *x*, *y*, *z*, and *d* are expressed in world units in the absolute coordinate system, while the coefficients *a*, *b*, and *c* are dimensionless.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the coefficient *d* of the laser plane equation.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_SPEED +		Inquires the speed of the object being scanned. Note that this value is negative if the object is moving away from the camera. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the speed, in world units per frame. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For profiling 3D reconstruction contexts set to M_DEPTH_CORRECTION, the InquireType parameter can be set to one

of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For profiling 3D reconstruction contexts set to M_DEPTH_CORRECTION																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_NUMBER_OF_COLUMNS +	Inquires the number of columns in the uncorrected depth map used for calibration. This corresponds to either the X-size or Y-size of the laser line images passed to the 3D reconstruction result buffer, depending on the M_SCAN_LANE_DIRECTION setting. This inquire type is only supported after a successful call to M3dmapCalibrate() or M3dmapCalibrateMultiple().															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of columns.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_COLUMNS_WITH_INVERSIONS +	Inquires the number of columns in the uncorrected depth map used for calibration, for which at least one inversion occurred. To determine the fraction of columns or rows of the camera image that were not ideally calibrated because of inversions, divide the value returned using M_NUMBER_OF_COLUMNS_WITH_INVERSIONS by M_NUMBER_OF_COLUMNS. This inquire type is only supported after a successful call to M3dmapCalibrate() or M3dmapCalibrateMultiple().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of columns.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_COLUMNS_WITH_MISSING_DATA +	Inquires the number of columns in the uncorrected depth map used for calibration, for which at least one calibration laser line was not detected. To determine the fraction of columns or rows of the camera image that were not ideally calibrated because of missing data, divide the value returned using M_NUMBER_OF_COLUMNS_WITH_MISSING_DATA by M_NUMBER_OF_COLUMNS. This inquire type is only supported after a successful call to M3dmapCalibrate() or M3dmapCalibrateMultiple().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the number of columns.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_INVERSIONS_PER_COLUMN +		Inquires the number of inversions, per column, in the uncorrected depth map used for calibration. This inquire type is only supported after a successful call to M3dmapCalibrate() or M3dmapCalibrateMultiple().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of inversions per column.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_MISSING_DATA_PER_COLUMN +		Inquires the number of missing data points per column in the uncorrected depth map used for calibration. This inquire type is only supported after a successful call to M3dmapCalibrate() or M3dmapCalibrateMultiple().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of missing data points per column.	c		e	f	g	a		i		j	k	l	m	n	b

For a point cloud container (3D reconstruction result buffer of type M_POINT_CLOUD_RESULT and LabelOrIndex set to M_GENERAL), the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a 3D reconstruction result buffer set to M_POINT_CLOUD_RESULT																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_NUMBER_OF_POINT_CLOUDS +		Inquires the number of point clouds in the specified 3D reconstruction result buffer.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the number of point clouds.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULTS_DISPLACEMENT_MODE +		Inquires the displacement mode, which determines how the 3D coordinates of a scanned object are returned with respect to the ongoing movement (displacement) of the conveyor. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_FIXED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CURRENT	Specifies that the results include the ongoing Y-axis displacement. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIXED	Specifies that the results do not include the ongoing Y-axis displacement. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULTS_DISPLACEMENT_Y +		Inquires the specified Y-axis displacement added to resulting 3D coordinates when M_RESULTS_DISPLACEMENT_MODE is set to M_FIXED. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-axis displacement. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For a point cloud (3D reconstruction result buffer of type M_POINT_CLOUD_RESULT and LabelOrIndex set to M_POINT_CLOUD_INDEX(), M_POINT_CLOUD_LABEL(), or M_ALL), the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a point cloud																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_CAMERA_LABEL_VALUE +	Inquires the camera label of the 3D reconstruction context associated with the specified point cloud. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 1023	Specifies the camera label value. The default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_LASER_LABEL_VALUE +	Inquires the laser label of the 3D reconstruction context associated with the specified point cloud. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

1 <= *Value* <= 2047	Specifies the laser label value. The default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_CLOUD_INDEX_VALUE +	Inquires the index of the point cloud with the specified label.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that there is no point cloud in the 3D reconstruction result buffer with the specified label.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Point cloud array index* <= 134217726	Specifies the index of the point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_CLOUD_LABEL_VALUE +	Inquires the label of the point cloud with the specified index. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that there is no point cloud in the 3D reconstruction result buffer with the specified index.	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Point cloud label* <= 134217727	Specifies the label of the point cloud.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For a point cloud; and for the following values: M_CALIBRATION_DEPTHS; M_NUMBER_OF_INVERSIONS_PER_COLUMN; M_NUMBER_OF_MISSING_DATA_PER_COLUMN.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																
InquireType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

For a draw 3D reconstruction context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j –

For a draw 3D reconstruction context																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_DRAW_ABSOLUTE_COORDINATE_SYSTEM +	Inquires whether to draw the absolute coordinate system's axes. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to draw the absolute coordinate system's axes. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to draw the absolute coordinate system's axes. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_CAMERA_COORDINATE_SYSTEM +	Inquires whether to draw the camera coordinate system's axes. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to draw the camera coordinate system's axes. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to draw the camera coordinate system's axes. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_CAMERA_COORDINATE_SYSTEM_NAME +	Inquires the name to draw for the camera coordinate system; returns "Camera" if no name was specified. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_COORDINATE_SYSTEM_LENGTH +	Inquires the drawn length of the specified coordinate system's axes. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the length (in world units) at which to draw the axes of the specified coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_FRUSTUM +		Inquires whether to draw the frustum. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to draw the frustum. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to draw the frustum. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_FRUSTUM_COLOR +		Inquires the frustum's color. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BLACK	Specifies the color black. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BLUE	Specifies the color blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BRIGHT_GRAY	Specifies the color bright gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_CYAN	Specifies the color cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_BLUE	Specifies the color dark blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_CYAN	Specifies the color dark cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_GREEN	Specifies the color dark green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_MAGENTA	Specifies the color dark magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_RED	Specifies the color dark red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_YELLOW	Specifies the color dark yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GRAY	Specifies the color gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GREEN	Specifies the color green. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR_LIGHT_BLUE	Specifies the color light blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_COLOR	Specifies no color. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_LASER_LINE_COORDINATE_SYSTEM +		Inquires whether to draw the laser line coordinate system's axes. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to draw the laser line coordinate system's axes.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to draw the laser line coordinate system's axes.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_LASER_PLANE +		Inquires how to draw the laser plane. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to draw the laser plane.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENABLE	Specifies to draw the laser plane.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_LASER_PLANE_COLOR_FILL +		Inquires the laser plane's fill color. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO_COLOR.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_COLOR	Specifies either the color red or the texture image. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BLACK	Specifies the color black. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BLUE	Specifies the color blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BRIGHT_GRAY	Specifies the color bright gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_CYAN	Specifies the color cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_BLUE	Specifies the color dark blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_CYAN	Specifies the color dark cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_GREEN	Specifies the color dark green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_MAGENTA	Specifies the color dark magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_RED	Specifies the color dark red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_YELLOW	Specifies the color dark yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GRAY	Specifies the color gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GREEN	Specifies the color green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_BLUE	Specifies the color light blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR_LIGHT_GRAY	Specifies the color light gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_COLOR	Specifies no color.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TEXTURE_IMAGE	Specifies to use the image passed to M3dmapDraw3d() with LaserPlaneTextureImageBufId, when drawing the laser plane. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_LASER_PLANE_COLOR_OUTLINE +		Inquires the laser plane's outline color. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_COLOR_WHITE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BLACK	Specifies the color black. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BLUE	Specifies the color blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BRIGHT_GRAY	Specifies the color bright gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_CYAN	Specifies the color cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_BLUE	Specifies the color dark blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR_DARK_CYAN	Specifies the color dark cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_GREEN	Specifies the color dark green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_MAGENTA	Specifies the color dark magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_RED	Specifies the color dark red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_YELLOW	Specifies the color dark yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GRAY	Specifies the color gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GREEN	Specifies the color green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_BLUE	Specifies the color light blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_COLOR	Specifies no color.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_LASER_PLANE_OPACITY +		Inquires the laser plane's opacity. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 20.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the laser plane's opacity.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_DRAW_CAMERA_COORDINATE_SYSTEM_NAME.

You can add the following value to the above-mentioned value to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For getting the string size																		
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_STRING_SIZE +		Retrieves the length of the string, including the terminating null character ("\0").															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

For a 3D reconstruction result buffer of type M_LASER_CALIBRATION_DATA, M_DEPTH_CORRECTED_DATA, or M_POINT_CLOUD_RESULT, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the maximum number of frames																		
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_MAX_FRAMES +		Inquires the maximum number of scanned laser lines that the result buffer keeps internally. When you specify a result buffer of type M_POINT_CLOUD_RESULT with LabelOrIndex set to M_GENERAL, this inquire type returns the maximum number of scanned laser lines that will be used for each subsequent point cloud created in this 3D reconstruction result buffer. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1024.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the maximum number of scanned laser lines to keep.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for

Tables:

- For inquiring about the system
- For 3D reconstruction contexts used for laser line profiling
- For profiling 3D reconstruction contexts set to M_CALIBRATED_CAMERA_LINEAR_MOTION
- For profiling 3D reconstruction contexts set to M_DEPTH_CORRECTION
- For a 3D reconstruction result buffer set to M_POINT_CLOUD_RESULT
- For a point cloud
- For a draw 3D reconstruction context
- For inquiring the maximum number of frames

the values listed in all the parameter association tables except **For a 3D alignment context**.

You can add the following value to the above-mentioned values to get the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value of an inquire type																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_DEFAULT	Inquires the default value of the specified inquire type.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about the system, For 3D reconstruction contexts used for laser line profiling, For profiling 3D reconstruction contexts set to M_CALIBRATED_CAMERA_LINEAR_MOTION, For profiling 3D reconstruction contexts set to M_DEPTH_CORRECTION, For a 3D reconstruction result buffer set to M_POINT_CLOUD_RESULT, For a point cloud, For a draw 3D reconstruction context, For inquiring the maximum number of frames; and for the following value: M_DRAW_CAMERA_COORDINATE_SYSTEM_NAME.

You can add one of the following values to the above-mentioned values to determine whether an inquire type is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether an inquire type is supported																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_HAS_DEFAULT	Inquires whether the specified inquire type has a default value.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b

M_SUPPORTED	Inquires whether the specified inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about the system, For 3D reconstruction contexts used for laser line profiling, For profiling 3D reconstruction contexts set to M_CALIBRATED_CAMERA_LINEAR_MOTION, For profiling 3D reconstruction contexts set to M_DEPTH_CORRECTION, For a 3D reconstruction result buffer set to M_POINT_CLOUD_RESULT, For a point cloud, For inquiring the maximum number of frames; and for the following values: M_DRAW_ABSOLUTE_COORDINATE_SYSTEM; M_DRAW_CAMERA_COORDINATE_SYSTEM; M_DRAW_COORDINATE_SYSTEM_LENGTH; M_DRAW_FRUSTUM; M_DRAW_FRUSTUM_COLOR; M_DRAW_LASER_LINE_COORDINATE_SYSTEM; M_DRAW_LASER_PLANE; M_DRAW_LASER_PLANE_COLOR_FILL; M_DRAW_LASER_PLANE_COLOR_OUTLINE; M_DRAW_LASER_PLANE_OPACITY; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested information to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_FLOAT* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested information to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID* **|** *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT16* **|** *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapRestore

Synopsis

Restore a 3D object from disk.

Syntax

```
AIL_ID M3dmapRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *M3dmapIdPtr          //out
)
```

Description

This function restores a 3D object that was previously saved to a file, using M3dmapSave() or M3dmapStream(). You can restore a 3D reconstruction context or 3D alignment context of types M_LASER or M_ALIGN_CONTEXT. You can also restore a 3D reconstruction result buffer or 3D alignment result buffer of types M_POINT_CLOUD_RESULT or M_ALIGN_RESULT.

This function restores all of the object's settings that were in effect when the object was saved. A restored 3D reconstruction context or 3D alignment context is ready to use, with no need to perform any calibration step.

When the restored 3D object is no longer required, release it using M3dmapFree() unless M_UNIQUE_ID was specified during restoration; if M_UNIQUE_ID was specified, the smart identifier manages the 3D object's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and directory of the file from which to restore the 3D object. The function handles (internally) the opening and closing of the file.

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") [1]	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, 3D object files have an M3D file extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the system on which to restore the 3D object.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

M3dmapIdPtr

Specifies the address of the variable in which to write the 3D object identifier or specifies the data type that the function should use to return the 3D object identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D object ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D object ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_3DMAP_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the 3D object (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the 3D alignment context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated 3D alignment context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the 3D alignment result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated 3D alignment result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the 3D reconstruction context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored 3D reconstruction context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the 3D reconstruction point cloud container result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored 3D reconstruction point cloud container result buffer. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the 3D object identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_3DMAP_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapSave

Synopsis

Save a 3D object to a file.

Syntax

```
void M3dmapSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID M3dmapId,             //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves to disk all the information about the previously allocated 3D object. You can save a 3D reconstruction context or 3D alignment context of types M_LASER or M_ALIGN_CONTEXT. You can also save a 3D reconstruction result buffer or 3D alignment result buffer of types M_POINT_CLOUD_RESULT or M_ALIGN_RESULT.

For a 3D reconstruction context or 3D alignment context, all camera or 3D profile sensor calibration information is saved, and the context will be ready to use on reload. You can reload the saved information, using M3dmapRestore() or M3dmapStream().

Parameters

FileName

Specifies the name and directory of the file in which to save the 3D object. The recommended extension is M3D.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, 3D object files have an M3D file extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

M3dmapId

Specifies the identifier of the 3D object to save.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapStream

Synopsis

Load, restore, or save a 3D object from/to a file or memory stream.

Syntax

```
void M3dmapStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *M3dmapIdPtr,           //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save a 3D object from/to a file or memory stream.

To inquire the number of bytes necessary to save a 3D object, you should first call this function (**M3dmapStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. In addition, any file saved using this function is equivalent to a file saved with M3dmapSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file. For a 3D reconstruction context or 3D alignment context, all camera or 3D profile sensor calibration information is saved, so the context will be ready to use on reload.

This function restores all of the object's settings that were in effect when the object was saved. A restored 3D object is ready to use, with no need to perform any camera or 3D profile sensor calibration steps.

Using **M3dmapStream**(), you can choose to save a backwards-compatible version of the 3D object, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore 3D objects saved using Aurora Imaging Library version 9.0 PP2 or above. Settings that do not exist in the lower version will be filled with default values when the 3D object is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the 3D object.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform on the 3D object.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the 3D object.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of stream																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the 3D object.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

M3dmapIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the 3D object.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the 3D object, in bytes.

> Note that the size of the 3D object varies depending on the Aurora Imaging Library version specified.

If the size is not required, you can set this parameter to **M_NULL**.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **M3dmapIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	MemPtrOrFileName - Possible values passed or returned															
	SysId															
	Version															
	M3dmapIdPtr - Possible values passed or returned															
M_INQUIRE_SIZE_BYTE		Inquires the number of bytes required to save a 3D object to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.														
	MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version	Specifies the Aurora Imaging Library version with which the 3D object must be compatible when saved. The size of a 3D object might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	M3dmapIdPtr	Specifies the address of the variable containing the identifier of the 3D object.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD		Loads the content of a specified file or memory stream into a previously allocated 3D object.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName	Specifies the file or memory stream from which to load the object.	c		e	f	g	a		i		j	k	l	m	n	b

		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
		AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. 3D objects typically have an M3D file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	M3dmapIdPtr		Specifies the address of the variable containing the identifier of the 3D object. All previous settings are overwritten with those of the loaded object, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the object is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE			Restores a 3D object from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName		Specifies the file or memory stream from which to restore the object.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

		AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. 3D objects typically have an M3D file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		Specifies the system on which to allocate the 3D object.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
		System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
	Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	M3dmapIdPtr		Specifies the address of the variable in which to write the identifier of the 3D object. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE			Saves a 3D object to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName		Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Aurora Imaging software products, when saving a 3D object to a file, use the M3D file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version		Specifies the Aurora Imaging Library version with which the 3D object must be compatible when saved. Note that the 3D object can only be up to one major release older than the current version.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
M3dmapIdPtr		Specifies the address of the variable containing the identifier of the 3D object to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.

M3dmapTriangulate

Synopsis

Perform stereo vision triangulation to find the 3D world coordinates of points from their pixel location in the image plane of two or more cameras.

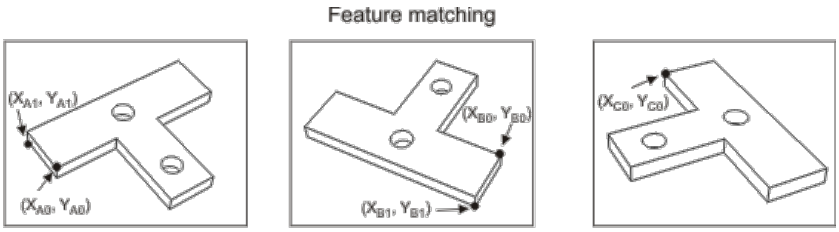
Syntax

```
void M3dmapTriangulate(
    const AIL_ID *ContextCalOrImageBufIdArrayPtr, //in
    const AIL_DOUBLE *PixelCoordXArrayPtr,        //in
    const AIL_DOUBLE *PixelCoordYArrayPtr,        //in
    AIL_DOUBLE *WorldCoordXArrayPtr,              //out
    AIL_DOUBLE *WorldCoordYArrayPtr,              //out
    AIL_DOUBLE *WorldCoordZArrayPtr,              //out
    AIL_DOUBLE *RMSErrorArrayPtr,                 //out
    AIL_INT NumCalibrations,                      //in
    AIL_INT NumPoints,                            //in
    AIL_INT64 CoordinateSystem,                   //in
    AIL_INT64 ControlFlag                         //in
)
```

Description

This function performs stereo vision triangulation (stereophotogrammetry) to calculate the X-, Y-, and Z-world coordinates of points (features) that can be seen by two or more cameras. You must identify the pixel location of the points in the image plane of the different cameras. Each of the camera setups must be calibrated in 3D mode using McalAlloc() with M_3D_ROBOTICS or M_TSAI_BASED so that the position of the cameras in the world is known. To calculate each point's location in the world, the point's location in each camera's image plane and the camera's location in the world are used.

With a single call to **M3dmapTriangulate**(), you can calculate the world coordinates of multiple points, even if a point does not appear in the image plane of all cameras. Each point must be seen by at least two cameras. To indicate that a point is not seen by a camera, use **M_INVALID_POINT**. For example, to determine the world coordinates of the following two points on an object that is seen by three cameras, you would pass the following arguments to **M3dmapTriangulate**():



Object as seen by camera A

Object as seen by camera B

Object as seen by camera C

Calibration Context	Point number	Array index	XPixelArrayPtr	YPixelArrayPtr
A	0	0	X_{A0}	Y_{A0}
	1	1	X_{A1}	Y_{A1}
B	0	2	X_{B0}	Y_{B0}
	1	3	X_{B1}	Y_{B1}
C	0	4	X_{C0}	Y_{C0}
	1	5	M_INVALID_POINT	M_INVALID_POINT

Array index	CalibrationIdArrayPtr
0	CalibrationContextA
1	CalibrationContextB
2	CalibrationContextC

> The camera calibration contexts of the camera setups must have been allocated in 3D mode using McalAlloc() with M_TSAI_BASED or M_3D_ROBOTICS and they must share a common world coordinate system. For more information about camera calibration, see Chapter 28: Calibrating your camera setup.

Although results can be obtained with only two cameras, rounding and extraction errors can affect the accuracy of the calculations. Using more than two cameras will increase the robustness of the function.

For points that could not be computed, their corresponding entries in the coordinate output arrays will be set to **M_INVALID_POINT** and their corresponding entries in the RMSErrorArrayPtr array will be set to 0.0. This can occur when all projection lines from the image planes to the point are perfectly parallel and therefore do not intersect.

Parameters

ContextCalOrImageBufIdArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:((NumCalibrations) * (NumPoints))
- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(NumCalibrations)

Specifies the address of an array containing the identifiers of the camera calibration contexts for each of the camera setups. Alternatively, the array can contain the identifiers of calibrated or corrected images; the operation uses the camera calibration contexts associated with the images.

The image buffers must be 1- or 3-band buffers.

PixelCoordXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:((NumCalibrations) * (NumPoints))
- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(NumCalibrations)

Specifies the address of the array containing the X-pixel coordinates of the common points in the image plane of each camera. This array must have a size equal to (NumCalibrations x NumPoints).

Specify the location of the points in the image plane of a given camera, before specifying them for another camera; list them in the same order for each camera. If more than two cameras are used and a point is seen by only a subset of all the cameras, indicate that the point is not seen by a camera by setting the corresponding entry in the array to **M_INVALID_POINT**. Order the groups of points according to order of their camera's calibration context in the specified list of camera calibration contexts (ContextCalOrImageBufIdArrayPtr).

PixelCoordYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:((NumCalibrations) * (NumPoints))
- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(NumCalibrations)

Specifies the address of the array containing the Y-pixel coordinates of the common points in the image plane of each camera. This array must have a size equal to (NumCalibrations x NumPoints).

Specify the location of the points in the image plane of a given camera, before specifying them for another camera; list them in the same order for each camera. If more than two cameras are used and a point is seen by only a subset

of all the cameras, indicate that the point is not seen by a camera by setting the corresponding entry in the array to **M_INVALID_POINT**. Order the groups of points according to order of their camera's calibration context in the specified list of camera calibration contexts (ContextCalOrImageBufIdArrayPtr).

WorldCoordXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:((NumCalibrations) * (NumPoints))
- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(NumCalibrations)

Specifies the address of an array in which to write the calculated X-world coordinates. This array must have a size equal to NumPoints.

WorldCoordYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:((NumCalibrations) * (NumPoints))
- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(NumCalibrations)

Specifies the address of the array in which to write the calculated Y-world coordinates. This array must have a size equal to NumPoints.

WorldCoordZArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:((NumCalibrations) * (NumPoints))
- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(NumCalibrations)

Specifies the address of the array in which to write the calculated Z-world coordinates. This array must have a size equal to NumPoints.

RMSErrorArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:((NumCalibrations) * (NumPoints))
- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(NumCalibrations)

Specifies the address of the array in which to write the root mean square (RMS) error for the calculated world coordinates of each point. This array must have a size equal to NumPoints.

If the RMS error is not needed, set this parameter to **M_NULL**.

NumCalibrations

Specifies the number of camera calibration contexts. This should correspond to the number of cameras. This parameter must be greater or equal to 2.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the ContextCalOrImageBufIdArrayPtr parameter.

NumPoints

Specifies the number of points to compute using stereo vision triangulation. This parameter must be greater or equal to 1.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the PixelCoordXArrayPtr or PixelCoordYArrayPtr parameter.

CoordinateSystem

Specifies the world coordinate system that is common to all the specified camera calibration contexts.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the common coordinate system																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ABSOLUTE_COORDINATE_SYSTEM	Specifies that the camera calibration contexts have the absolute coordinate system in common.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_COORDINATE_SYSTEM	Specifies that the camera calibration contexts have the relative coordinate system in common.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROBOT_BASE_COORDINATE_SYSTEM	Specifies that the camera calibration contexts have the robot base coordinate system in common.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOOL_COORDINATE_SYSTEM	Specifies that the camera calibration contexts have the tool coordinate system in common.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmap11.lib.
DLL	Requires ail11.dll; ail3dmap11.dll.